

# PPS - Scala Stream Fusion

Leonardo Naddei

27 settembre 2026

# Indice

|          |                                                                |           |
|----------|----------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduzione</b>                                            | <b>3</b>  |
| 1.1      | Obiettivi e criteri di valutazione . . . . .                   | 4         |
| <b>2</b> | <b>Analisi dei requisiti</b>                                   | <b>5</b>  |
| 2.1      | Requisiti di business . . . . .                                | 5         |
| 2.2      | Requisiti funzionali utente . . . . .                          | 5         |
| 2.3      | Requisiti funzionali di sistema . . . . .                      | 6         |
| 2.4      | Requisiti negativi ed esclusioni (Out of Scope) . . . . .      | 6         |
| 2.5      | Requisiti non funzionali . . . . .                             | 7         |
| <b>3</b> | <b>Background e tecnologie abilitanti</b>                      | <b>8</b>  |
| 3.1      | Stream fusion . . . . .                                        | 8         |
| 3.1.1    | Push stream . . . . .                                          | 8         |
| 3.1.2    | Pull stream . . . . .                                          | 9         |
| 3.2      | Metaprogrammazione in Scala 3 . . . . .                        | 9         |
| 3.2.1    | Introduzione alla Multi-stage programming in Scala 3 . . . . . | 9         |
| 3.3      | Metaprogrammazione generativa e analitica . . . . .            | 9         |
| 3.3.1    | La reflection API e l'AST tipizzato . . . . .                  | 10        |
| 3.3.2    | Tipi staged e tipi riflessivi . . . . .                        | 10        |
| 3.3.3    | Beta reduction delle funzioni . . . . .                        | 11        |
| 3.4      | Definizione di variabili . . . . .                             | 11        |
| <b>4</b> | <b>Design architetturale</b>                                   | <b>12</b> |
| 4.1      | Architettura generale . . . . .                                | 12        |
| 4.2      | Componenti del sistema . . . . .                               | 12        |
| 4.3      | Architettura del compilatore . . . . .                         | 13        |
| <b>5</b> | <b>Design di dettaglio</b>                                     | <b>14</b> |
| 5.1      | Design dell'API pubblica . . . . .                             | 14        |
| 5.1.1    | Operazioni terminali . . . . .                                 | 14        |
| 5.1.2    | Creazione dello stream: <code>from</code> . . . . .            | 15        |
| 5.1.3    | Tipologie di stream . . . . .                                  | 15        |
| 5.1.4    | Operazione terminale <code>collect</code> . . . . .            | 16        |
| 5.1.5    | Configurazione compile-time e runtime . . . . .                | 17        |
| 5.1.6    | Import semplificato dell'API . . . . .                         | 17        |
| 5.2      | Design della Stream IR . . . . .                               | 17        |
| 5.2.1    | Struttura dell'albero . . . . .                                | 18        |
| 5.3      | Rappresentazione per la Code Generation (OP Ir) . . . . .      | 18        |
| 5.3.1    | Valori . . . . .                                               | 19        |
| 5.3.2    | Operazioni . . . . .                                           | 20        |
| 5.4      | La collection strategy . . . . .                               | 20        |

|          |                                                        |           |
|----------|--------------------------------------------------------|-----------|
| <b>6</b> | <b>Implementazione</b>                                 | <b>21</b> |
| 6.1      | Parsing                                                | 21        |
| 6.1.1    | Parsing della stream                                   | 21        |
| 6.1.2    | Parsing del collector                                  | 22        |
| 6.1.3    | Parsing della configurazione statica                   | 23        |
| 6.2      | Enrichment                                             | 23        |
| 6.2.1    | Materializzazione delle espressioni                    | 24        |
| 6.2.2    | Stato e condizioni di terminazione                     | 24        |
| 6.2.3    | Analisi della cardinalità                              | 24        |
| 6.3      | Traduzione in OP IR                                    | 25        |
| 6.3.1    | Selezione della strategia terminale                    | 25        |
| 6.3.2    | Materializzazione del risultato                        | 25        |
| 6.3.3    | Generazione della pipeline e delle sorgenti            | 26        |
| 6.3.4    | Generazione del piano parallelo                        | 27        |
| 6.4      | Lowering                                               | 27        |
| 6.4.1    | Traduzione dell'OP IR                                  | 27        |
| 6.4.2    | Applicazione e inlining delle funzioni                 | 28        |
| 6.4.3    | Lowering del parallelismo                              | 28        |
| 6.5      | Esempio di traduzione end-to-end                       | 29        |
| 6.5.1    | L'input alla libreria                                  | 29        |
| 6.5.2    | Il risultato del parsing                               | 31        |
| 6.5.3    | L'output dell'enrichment                               | 31        |
| 6.5.4    | Generazione dell'OpIR                                  | 32        |
| 6.5.5    | Lowering verso l'AST Scala                             | 33        |
| 6.5.6    | Il bytecode JVM generato                               | 34        |
| <b>7</b> | <b>Testing e valutazione delle prestazioni</b>         | <b>35</b> |
| 7.1      | Testing                                                | 35        |
| 7.2      | Valutazione delle prestazioni                          | 36        |
| 7.2.1    | Metodologia                                            | 36        |
| 7.2.2    | Benchmark sequenziali                                  | 36        |
| 7.2.3    | Benchmark paralleli                                    | 37        |
| 7.2.4    | Comportamento a freddo e livelli di compilazione JIT   | 38        |
| 7.2.5    | Impatto delle closure: il caso delle Capturing Lambdas | 39        |
| 7.2.6    | Allocazioni e comportamento del Garbage Collector      | 40        |
| <b>8</b> | <b>Retrospectiva</b>                                   | <b>42</b> |
| 8.1      | Limiti e compromessi progettuali                       | 42        |
| 8.2      | Sviluppi futuri                                        | 42        |
| 8.3      | Conclusioni                                            | 43        |

# Capitolo 1

## Introduzione

Le API basate su pipeline costituiscono un modello di programmazione diffuso per l'elaborazione di collezioni e sequenze di dati. Operazioni come `map`, `filter` e `flatMap` permettono di descrivere una computazione in forma dichiarativa, separando la logica dell'elaborazione dai dettagli relativi all'attraversamento della struttura dati.

Questa forma di astrazione migliora la leggibilità e la composizionalità del codice, ma può introdurre un costo rispetto a un'implementazione imperativa specializzata. Sulla JVM, una pipeline ad alto livello può richiedere la costruzione di iteratori o strutture intermedie, l'allocazione di closure, l'invocazione indiretta delle funzioni associate agli operatori e, in alcuni casi, operazioni di boxing e unboxing. Il compilatore JIT può eliminare parte di questi costi, ma la sua efficacia dipende dalla struttura del programma.

L'impatto di tali overhead è osservabile anche in sistemi reali. Nel progetto Apache Cassandra sono state, ad esempio, introdotte ottimizzazioni mirate alla rimozione delle Java Stream API da percorsi di esecuzione particolarmente frequenti [4] e all'eliminazione di capturing lambda responsabili di allocazioni [3]. Questi interventi mostrano come possa risultare necessario rinunciare ad astrazioni di alto livello per ottenere un'esecuzione più efficiente.

Un ulteriore limite dei sistemi basati sull'applicazione dinamica di lambda function emerge quando lo stesso call site viene utilizzato con implementazioni differenti. In questi casi il sito di chiamata può diventare polimorfo o megamorfo. Il problema è noto e sono state proposte diverse strategie per preservare e propagare maggiori informazioni di specializzazione attraverso i confini delle chiamate. Tra queste, la proposta della *Parametric VM* discute meccanismi per associare informazioni di specializzazione a metodi e call site, consentendo alla JVM di generare codice maggiormente specializzato [7]. *FusedStream* evita di affrontare questo problema a runtime: quando la funzione è nota staticamente, ne effettua l'inlining e la monomorfizzazione a compile-time, eliminando il relativo dispatch dal codice generato.

Il problema affrontato in questo progetto consiste quindi nel ridurre il divario tra due modalità di esprimere la stessa computazione: da un lato una pipeline dichiarativa e composizionale, dall'altro un ciclo imperativo esplicitamente specializzato per l'elaborazione da eseguire.

L'obiettivo di *FusedStream* è preservare il primo modello di programmazione producendo, per quanto possibile, codice eseguibile vicino al secondo. La libreria espone un'API ispirata alle comuni API per stream e collezioni ma, diversamente da una normale implementazione runtime, la pipeline non viene rappresentata durante l'esecuzione come una successione di oggetti e chiamate. La sua struttura viene invece analizzata durante la compilazione mediante le funzionalità di metaprogrammazione di Scala 3 e trasformata in codice specializzato rispetto alla specifica composizione di sorgente, operatori e operazione terminale.

Il principio alla base della trasformazione è quello della *stream fusion*, introdotto da Coutts et al. in *Stream Fusion: From Lists to Streams to Nothing at All* [1]: le operazioni intermedie vengono fuse in un singolo flusso di esecuzione, evitando, quando possibile, la materializzazione dei risultati intermedi. L'analisi statica della pipeline permette inoltre di sfruttare informazioni

relative ai tipi, alla sorgente, alla cardinalità del risultato e alla modalità di esecuzione per selezionare differenti strategie di generazione del codice.

Per certi aspetti, il modello ottenuto richiama la monomorfizzazione a compile-time dei generics adottata da Rust e la specializzazione dei template C++: il codice viene specializzato rispetto ai tipi concretamente utilizzati. FusedStream estende questo principio alla struttura stessa della computazione: le macro di Scala 3 analizzano la composizione della pipeline e generano un'implementazione specializzata rispetto alla sorgente, agli operatori e all'operazione terminale.

Particolare attenzione è stata dedicata anche alla progettazione dell'API. Le proprietà e i vincoli della pipeline vengono, quando possibile, rappresentati attraverso il sistema di tipi anziché rimandarne la verifica alla fase di analisi o di esecuzione. La distinzione tra stream sequenziali e paralleli, le caratteristiche delle sorgenti e i requisiti delle operazioni terminali vengono quindi utilizzati per impedire già in fase di compilazione combinazioni non supportate.

Le operazioni terminali sono modellate attraverso un'astrazione estensibile basata sui *collector*. Il sistema permette sia di utilizzare collector definiti dall'utente, sia di riconoscere alcune operazioni note alla libreria e sostituirle con strategie di generazione specializzate. Lo stesso modello viene esteso all'esecuzione parallela: quando sorgente, operatori e terminale possiedono le proprietà necessarie, la computazione può essere partizionata in più unità di lavoro alle quali viene applicata la medesima pipeline fusa.

## 1.1 Obiettivi e criteri di valutazione

L'obiettivo del progetto è verificare se sia possibile preservare l'espressività di una pipeline dichiarativa generando codice con caratteristiche prossime a quelle di un'implementazione imperativa specializzata. La valutazione considera quindi tre aspetti principali: la *correttezza semantica* della trasformazione, le *caratteristiche del codice generato* e le *prestazioni a runtime*.

La correttezza viene verificata mediante test funzionali e di compilazione. L'ispezione del codice e del bytecode generato permette invece di verificare l'effettiva fusione delle operazioni, la specializzazione delle chiamate e l'eliminazione di allocazioni e dispatch non necessari. Infine, benchmark dedicati confrontano il throughput con implementazioni imperative equivalenti e con le corrispondenti pipeline offerte dalle librerie standard.

## Capitolo 2

# Analisi dei requisiti

### 2.1 Requisiti di business

- RB-1**      **Colmare il divario tra espressività e prestazioni.** La programmazione funzionale favorisce la leggibilità, l'astrazione e la manutenibilità del codice attraverso l'impiego di pipeline dichiarative. Tuttavia, sulla JVM tali astrazioni comportano comunemente una penalizzazione prestazionale rispetto a cicli imperativi scritti a mano. L'obiettivo primario del progetto è annullare questo compromesso, consentendo di scrivere codice ad alto livello dichiarativo che compili direttamente in codice a basso livello massimamente efficiente.
- RB-2**      **Adozione a basso attrito.** L'integrazione della libreria all'interno di progetti esistenti non deve richiedere modifiche al toolchain standard del compilatore né l'apprendimento di API complesse o esotiche. La sintassi offerta deve risultare familiare a chi già utilizza le collezioni standard di Scala o le API di Java Stream, garantendo un'interfaccia compatta, uniforme e accessibile tramite un singolo import.

### 2.2 Requisiti funzionali utente

- RFU-1**      **Creazione dello Stream.** L'utente deve poter istanziare un flusso di elaborazione a partire da diverse sorgenti di dati (ad esempio collezioni Scala, array primitivi o iteratori) attraverso un unico punto di ingresso intuitivo (`FusedStream.from(...)`).
- RFU-2**      **Configurazione.** L'utente deve poter sovrascrivere il comportamento predefinito del sistema, sia per quanto riguarda i parametri di generazione statica del codice che per i parametri riguardanti l'esecuzione a runtime. In assenza di configurazione esplicita, il sistema deve utilizzare valori predefiniti senza richiedere ulteriori interventi da parte dell'utente.
- RFU-3**      **Esecuzione parallela esplicita.** Qualora la sorgente dei dati sia adatta, l'utente deve poter richiedere l'esecuzione concorrente della pipeline invocando esplicitamente il metodo `.parallel()`.
- RFU-4**      **Raccolta dei risultati.** L'utente deve poter raccogliere il risultato di una stream tramite `.collect(...)`, potendo utilizzare Collector predefiniti della libreria (come `toList`, `toArray`, `summing`, `findFirst`) e Collector personalizzati che rispettino l'interfaccia di accumulazione estendibile fornita dal framework.

- RFU-5 Guida contestuale tramite Type System.** L'API deve guidare attivamente l'utente durante la costruzione della pipeline sfruttando il type system. Tramite tipi e subtyping avanzato, l'IDE e il compilatore devono esporre ed accettare solo ed esclusivamente le operazioni valide per lo stato corrente dello stream (es. inibendo metodi non supportati in parallelo o impedendo l'uso di collector incompatibili con la policy di terminazione).

## 2.3 Requisiti funzionali di sistema

- RFS-1 Riconoscimento e analisi della pipeline statica.** Il sistema deve essere in grado di ispezionare l'albero sintattico di una pipeline dichiarativa e scomporla nella sequenza logica di sorgente, trasformazioni intermedie e operazione terminale a compile time.
- RFS-2 Fusione delle operazioni sequenziali.** Il sistema deve collassare le trasformazioni in un unico flusso di esecuzione continuo, eliminando la materializzazione di collezioni o iteratori tra uno stadio e il successivo.
- RFS-3 Generazione di codice specializzato.** Il sistema deve emettere codice imperativo target direttamente specializzato per i tipi di dato coinvolti e per l'operazione terminale richiesta, evitando chiamate polimorfiche o indirette.
- RFS-4 Gestione del flusso e terminazione anticipata.** Il sistema deve regolare il flusso di controllo del ciclo generato per supportare sia l'elaborazione esaustiva dell'intero flusso sia l'interruzione anticipata (*short-circuiting*) quando la logica dell'operazione terminale o di filtro lo richiede.
- RFS-5 Composizione di flussi annidati.** Il sistema deve consentire la fusione di flussi generati dinamicamente da elementi intermedi (`flatMap`), integrando il ciclo interno nel contesto del ciclo esterno senza creare buffer di accumulazione intermedi o computazioni superflue.
- RFS-6 Supporto all'esecuzione parallela.** Per le sorgenti e i terminali compatibili, il sistema deve emettere codice in grado di suddividere il carico di lavoro su più worker e aggregarne i risultati intermedi.

## 2.4 Requisiti negativi ed esclusioni (Out of Scope)

Al fine di garantire la terminazione del metaprogramma, la predicibilità prestazionale e la robustezza del compilatore, sono definiti i seguenti requisiti negativi:

- RN-1 Assenza di composizione dinamica runtime.** Il sistema non deve supportare la costruzione dinamica di pipeline determinate da rami condizionali ordinari a runtime (ad esempio `if (cond) s.map(...) else s.filter(...)`). L'intera catena di trasformazioni deve essere fissa e nota sintatticamente al call-site di compilazione.
- RN-2 Nessuna ottimizzazione opaca sul codice utente.** Il sistema non deve alterare né riscrivere la logica interna delle espressioni arbitrarie fornite dall'utente all'interno delle lambda, limitandosi all'inlining del corpo nel call-site generato.

- RN-3**      **Nessun supporto a flussi annidati di stream.** Il sistema non supporta la generazione di stream i cui singoli elementi siano a loro volta istanze di stream tramite trasformazioni mapping (es. `map(x => FusedStream.of(x))`). La sola composizione nested consentita è quella finalizzata all'appiattimento esplicito tramite `flatMap`.
- RN-4**      **Nessun fall-back silente a runtime.** Se una composizione di stream viola i vincoli di tipo o non è compatibile con la fusione, il compilatore deve rigettare esplicitamente il programma tramite un errore di tipo statico o errore di compilazione della macro, senza ricorrere a conversioni lente o fallback trasparenti a runtime.
- RN-5**      **Nessun inlining o ottimizzazione per funzioni non-lambda.** Qualora a una trasformazione (es. `map`) venga passato una funzione preesistente anziché un'espressione lambda, il compilatore non applica la  $\beta$ -reduction né l'inlining statico.

## 2.5 Requisiti non funzionali

- RNF-1**      **Prestazioni a runtime.** Il codice imperativo generato deve esibire un throughput computazionale comparabile a cicli imperativi scritti manualmente in Java, riducendo l'overhead computazionale rispetto alle pipeline funzionali convenzionali ed azzerando il boxing introdotto dalla libreria per i tipi primitivi.
- RNF-2**      **Assenza di allocazioni intermedie.** L'elaborazione delle operazioni intermedie stateless non deve allocare oggetti effimeri sul runtime heap (nessun iteratore, nessuna closure associata a lambda, nessun contenitore intermedio).
- RNF-3**      **Preservazione della semantica.** A parità di sorgente, funzioni utente e configurazione, il codice generato deve produrre risultati semanticamente equivalenti a quelli definiti dalla pipeline dichiarativa, preservando l'ordine delle operazioni, la cardinalità del flusso e la semantica delle operazioni terminali.



## Capitolo 3

# Background e tecnologie abilitanti

Questo capitolo introduce i principali concetti teorici e gli strumenti di Scala 3 necessari alla comprensione delle scelte progettuali e implementative adottate in FusedStream.

### 3.1 Stream fusion

Una pipeline di stream può essere rappresentata secondo due modelli di esecuzione fondamentali: un modello *push*, guidato dal produttore, e un modello *pull*, guidato dal consumatore. Questa distinzione viene discussa, nel contesto della stream fusion, da Kiselyov et al. in *Stream Fusion, to Completeness* [2].

#### 3.1.1 Push stream

Nel modello *push* è la sorgente a guidare l'elaborazione. Ogni elemento prodotto viene immediatamente propagato attraverso gli operatori successivi della pipeline fino a raggiungere il terminale. Concettualmente, l'intera sequenza di operatori può quindi essere vista come una “mega-funzione” applicata a ciascun elemento della sorgente.

Per esempio, una pipeline della forma

```
1 stream
2 .filter(p)
3 .map(f)
4 .collect(collector)
```

può essere interpretata, come la composizione di una singola funzione:

$$\text{consume}(x) = \text{collector}(\text{map}(\text{filter}(x))).$$

Questa rappresentazione rende particolarmente semplice l'implementazione di operatori come **map** e **filter**: ciascun operatore scarta o inoltra (eventualmente trasformandolo) l'elemento ricevuto senza modificare il meccanismo con cui la sorgente viene attraversata.

E' più complessa la gestione di operatori che modificano la cardinalità della stream. Operazioni come **skip** e **limit** richiedono infatti di mantenere uno stato relativo al numero di elementi osservati. Nel caso di **limit**, una volta raggiunto il numero di elementi, è necessario interrompere anticipatamente la produzione della sorgente.

Una difficoltà ancora maggiore emerge con operatori che combinano più stream come lo **zip**. Nel modello push ciascuna sorgente possiede infatti il proprio flusso di produzione: per associare l'elemento corrente di una stream con quello corrispondente di un'altra è necessario introdurre un meccanismo esplicito di sincronizzazione tra i due produttori.

### 3.1.2 Pull stream

Nel modello *pull*, al contrario, il controllo dell'esecuzione è affidato al consumatore. La stream viene rappresentata come un oggetto capace di produrre, su richiesta, il proprio elemento successivo. In questo caso non è la sorgente a inviare gli elementi verso gli operatori successivi, ma è il terminale a richiederli risalendo progressivamente la pipeline. Ogni operatore intermedio definisce quindi il proprio comportamento in termini dell'operatore precedente: una `map`, ad esempio, richiede un elemento alla stream sottostante e applica ad esso la funzione di trasformazione; un `filter` continua invece a richiedere elementi finché non ne incontra uno che soddisfa il predicato.

Questa organizzazione rende naturali operazioni come `limit`, poiché il consumatore può semplicemente cessare di richiedere nuovi elementi, e soprattutto operatori come `zip`: è sufficiente richiedere contemporaneamente un elemento a ciascuna delle due stream e combinarne i risultati. Il prezzo da pagare è una rappresentazione più esplicita dello stato dell'iterazione e, in assenza di successive ottimizzazioni, un maggior numero di passaggi intermedi necessari alla produzione di ciascun elemento.

## 3.2 Metaprogrammazione in Scala 3

La realizzazione del compilatore di FusedStream richiede sia la generazione di nuovo codice sia l'analisi e la trasformazione del codice ricevuto dalla macro. Questa sezione introduce i meccanismi di metaprogrammazione di Scala 3 utilizzati a tale scopo. Quanto segue e gli esempi riportati fanno riferimento al modello di metaprogrammazione generativa e analitica presentati in [8, 9]

### 3.2.1 Introduzione alla Multi-stage programming in Scala 3

Le due primitive fondamentali del sistema di metaprogrammazione staged di Scala 3 sono le *quote* e gli *splice*. Una quote `'{ expression }` costruisce una rappresentazione tipizzata del codice contenuto al suo interno. Se `expression` produce un valore di tipo `T`, il risultato della quote ha tipo `Expr[T]` e rappresenta codice che produrrà un valore di tipo `T` nello stage successivo.

Uno splice: `$(expression)` opera invece nella direzione opposta e inserisce all'interno del codice quotato il frammento di programma rappresentato da un'espressione `Expr[T]`.

Un esempio di generazione staged è l'elevamento a potenza con esponente noto a compile time:

```
1 def powCode(x: Expr[Int], n: Int)(using Quotes): Expr[Int] =
2   if n == 0 then
3     Expr(1)
4   else
5     '{ $x * ${ powCode(x, n - 1) } }
```

La ricorsione di `powCode` viene eseguita durante l'espansione della macro, per un esponente pari a tre, il frammento generato è equivalente a `x * x * x * 1` e non contiene quindi cicli o ricorsioni.

## 3.3 Metaprogrammazione generativa e analitica

Una macro generativa costruisce un nuovo frammento di programma a partire da informazioni note durante la compilazione. E' è tuttavia necessario anche compiere l'operazione inversa: ricevere un frammento di programma, riconoscerne la struttura e decomporlo nei suoi componenti.

Scala 3 supporta questa forma di metaprogrammazione in maniera elementare attraverso il pattern matching sul codice quotato. Una generica espressione può essere analizzata con un pattern della forma

```
1 expr match {  
2   case '{ someFunction($argument) } =>  
3     ...  
4 }
```

Nel progetto, tuttavia, è necessario accedere a dettagli più fini dell'albero generato dal compilatore, rendendo necessario il passaggio alla Reflection API.

### 3.3.1 La reflection API e l'AST tipizzato

Il tipo `Expr[T]` offre un'interfaccia ad alto livello sul codice. Quando è necessario analizzare direttamente il programma è possibile convertire una `Expr[T]` nel relativo nodo dell'AST del compilatore con `expr.asTerm`. La Reflection API espone la struttura tipizzata prodotta dal compilatore, tra i nodi più importanti utilizzati nel parser vi sono:

- `Apply`, per l'applicazione di una funzione;
- `TypeApply`, per un'applicazione contenente parametri di tipo;
- `Select`, per la selezione di un membro;
- `Ident`, per il riferimento a un identificatore;
- `Block`, per un blocco contenente dichiarazioni e un'espressione finale;
- `Inlined`, per rappresentare il risultato di un'espansione inline;
- `ValDef`, `Assign` e `Ref`, utilizzati durante la generazione di nuovo codice.

### 3.3.2 Tipi staged e tipi riflessivi

A livello di tipi Scala 3 mette a disposizione due rappresentazioni:

`Type[T]` rappresenta il tipo `T` nell'API staged ed è utilizzabile insieme a `Expr[T]`, quote e splice;

`TypeRepr` rappresenta invece un tipo all'interno della Reflection API e permette di ispezionarne direttamente la struttura.

Data una `TypeRepr`, il metodo `.asType` restituisce un valore `Type[?]` sul quale è possibile eseguire pattern matching:

```
1 item.asType match {  
2   case '[t] => ...  
3 }
```

Un esempio del suo utilizzo è l'estrazione del tipo per la creazione di un iteratore da un singolo elemento:

```
1 private def createSourceOf(item: Term): ParsedTree = {  
2   item.asType match {  
3     case '[t] =>  
4       val singleExpr = item.asExprOf[t]  
5       val iterableExpr: Expr[Iterable[t]] =
```

```

6         '{ Iterable.single[t]($ { singleExpr }) }'
7     ...
8 }
9 }

```

Questo esempio contiene in pochi passaggi l'intero collegamento tra reflection e staged programming:

1. viene determinato il tipo del termine dell'AST ricevuto;
2. il tipo viene estratto in `t` mediante il pattern `'[t]`;
3. il `Term` originale viene convertito in `Expr[t]`;
4. viene generato un nuovo frammento `Expr[Iterable[t]]`;

### 3.3.3 Beta reduction delle funzioni

Un altro elemento importante è la funzionalità di  $\beta$ -reduction. Data una funzione:

$$f = x \mapsto e, \quad (3.1)$$

L'applicazione

$$f(a) \quad (3.2)$$

Può essere riscritta sostituendo direttamente  $x$  con  $a$  all'interno di  $e$ . Scala 3 rende disponibile questa trasformazione attraverso: `Expr.betaReduce(...)`. Incorporando staticamente il corpo della lambda nell'IR generato, la macro elimina sia la rappresentazione della funzione come entità invocabile sia il relativo dispatch.

## 3.4 Definizione di variabili

Nell'AST di Scala, un riferimento a una variabile è associato al relativo `Symbol`. Il nome è solamente una delle proprietà del simbolo e non ne rappresenta l'identità. La Reflection API permette di creare nuove variabili attraverso: `Symbol.newVal(...)`, e di fare successivamente riferimento allo stesso simbolo attraverso `Ref(symbol)`. La definizione viene creata nell'AST tramite `ValDef(symbol, Some(initializer))`. La distinzione tra nome e simbolo garantisce inoltre l'*hygiene* della macro: variabili generate dalla macro non possono andare in conflitto con variabili omonime definite nel codice utente.

# Capitolo 4

## Design architetturale

### 4.1 Architettura generale

FusedStream è organizzata in due aree principali: l'*API pubblica* e il *compilatore*.

L'API pubblica, descritta nella Sezione 5.1, espone il modello con cui l'utente definisce sorgenti, operatori intermedi e operazioni terminali. L'invocazione di un'operazione terminale attiva il compilatore, che analizza staticamente la pipeline e genera il codice specializzato necessario alla sua esecuzione.

Il sistema distingue quindi nettamente le due fasi del ciclo di vita della pipeline: a compile time ne viene analizzata la struttura e viene prodotto il codice corrispondente. A runtime viene eseguito direttamente il codice generato, senza interpretare nuovamente la descrizione della pipeline.

### 4.2 Componenti del sistema

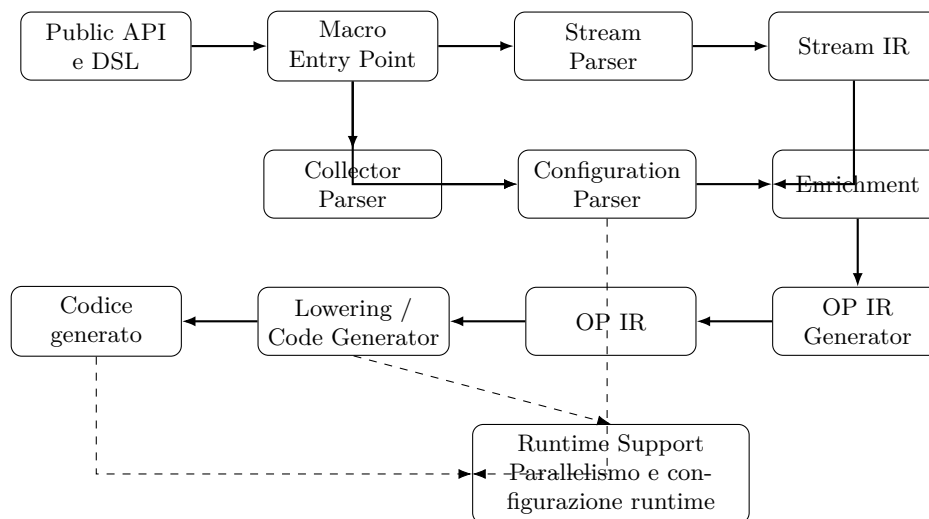


Figura 4.1: Architettura generale e principali componenti del sistema. Le frecce continue rappresentano il flusso di compilazione della pipeline, mentre le frecce tratteggiate indicano le dipendenze dal supporto runtime.

Come mostrato in Figura 4.1, il flusso di compilazione parte dalla Public API e termina con la produzione del codice eseguibile. Il *Macro Entry Point* costituisce il punto di ingresso del compilatore e orchestra le diverse fasi di trasformazione.

Il *Parser* analizza l'AST Scala della pipeline e ne ricostruisce la rappresentazione logica nella *Stream IR*. Parallelamente, il collector e la configurazione vengono analizzati per determinare rispettivamente la strategia terminale e i parametri che influenzano la generazione e l'esecuzione del codice.

La *Stream IR* viene successivamente completata dalla fase di *Enrichment*, che associa alla pipeline le informazioni statiche necessarie alla pianificazione dell'esecuzione, tra cui proprietà della sorgente, cardinalità e condizioni di terminazione. L'*OP IR Generator* utilizza questa rappresentazione arricchita per produrre la *Operation IR* (OP IR), nella quale le operazioni logiche della pipeline vengono trasformate in un piano orientato all'esecuzione.

Infine, il *Lowering* traduce la OP IR in un nuovo AST Scala, che viene restituito al compilatore e reinserito nel normale processo di compilazione. Il *Runtime Support*, esterno al flusso principale di trasformazione, fornisce invece le funzionalità richieste dal codice generato, in particolare per l'esecuzione parallela e la configurazione runtime.

### 4.3 Architettura del compilatore

La pipeline interna del compilatore può essere suddivisa concettualmente in *frontend* e *backend*, come mostrato in Figura 4.2.

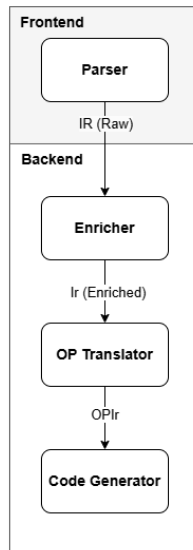


Figura 4.2: Architettura generale del compilatore di FusedStream.

Il *frontend* è responsabile dell'interpretazione della DSL: analizza l'AST Scala ricevuto dalla macro e ricostruisce la struttura della pipeline nella *Stream IR*. Questa rappresentazione mantiene una corrispondenza diretta con i concetti del dominio, quali sorgenti e operatori intermedi.

Il *backend* trasforma progressivamente questa rappresentazione fino a ottenere il codice eseguibile. L'*Enrichment* completa la *Stream IR* con le informazioni statiche necessarie alla generazione; l'*OP IR Generator* traduce quindi la pipeline arricchita in una rappresentazione orientata all'esecuzione; infine, il *Lowering* converte la OP IR nel nuovo AST Scala.

Il flusso complessivo del compilatore può essere sintetizzato come:

Scala AST  $\longrightarrow$  Stream IR<sub>Raw</sub>  $\longrightarrow$  Stream IR<sub>Enriched</sub>  $\longrightarrow$  OP IR  $\longrightarrow$  Generated Scala AST.

# Capitolo 5

## Design di dettaglio

### 5.1 Design dell'API pubblica

Particolare attenzione è stata posta nella progettazione dell'API pubblica della libreria. L'obiettivo è stato quello di fornire un'interfaccia che garantiscesse contemporaneamente:

- la possibilità di descrivere pipeline sufficientemente espressive
- l'estendibilità, in particolare per quanto riguarda le operazioni terminali
- il maggior numero possibile di garanzie a compile time
- la minimizzazione del codice necessario per verificare dinamicamente la correttezza delle combinazioni di operazioni richieste dall'utente.

Dal punto di vista concettuale, ogni stream può essere scomposto in tre componenti principali:

1. una sorgente, generalmente costituita da una collezione o da un array;
2. una sequenza di operazioni intermedie (`map`, `filter`, `flatMap`,...)
3. un'operazione terminale

La gerarchia di tipi esposta dall'API riflette direttamente questa struttura. Tramite di `trait sealed`, tipi parametrici, tipi opachi e vincoli sui parametri di tipo è stato possibile modellare le proprietà della pipeline. In questo modo il compilatore Scala può guidare l'utente durante la costruzione dello stream e impedire le combinazioni non supportate.

Allo stesso tempo, l'API mantiene un certo grado di estendibilità, in particolare consentendo all'utente di definire nuove operazioni terminali attraverso l'interfaccia del `Collector`.

#### 5.1.1 Operazioni terminali

Le operazioni terminali sono rappresentate attraverso l'astrazione `Collector`. La gerarchia completa delle astrazioni coinvolte è riportata in Figura 5.2.

I quattro parametri di tipo rappresentano rispettivamente:

- E: il tipo degli elementi ricevuti dallo stream;
- B: il tipo dello stato intermedio, o buffer, utilizzato durante l'accumulazione;
- R: il tipo del risultato finale prodotto dal collector;
- S: la politica di terminazione del collector.

Il metodo `supplier` crea lo stato iniziale dell'accumulator, `accumulator` aggiunge un nuovo elemento, il valore booleano restituito segnala che l'elaborazione può essere interrotta (si pensi a una `findFirst`). Infine, `finisher` trasforma lo stato intermedio nel risultato finale.

I collector possono essere esaustivi ovvero richiedere l'elaborazione dell'intera sorgente, oppure possono terminare anticipatamente senza consumare tutti i dati. Questa proprietà è rappresentata tramite `TerminationPolicy`, i cui sottotipi `Exhaustive` e `ShortCircuiting` distinguono rispettivamente i collector che richiedono l'elaborazione completa della sorgente da quelli che possono terminare anticipatamente (Figura 5.2).

## Collector combinabili e parallelismo

Per poter eseguire una riduzione in parallelo non è sufficiente che il collector sia `Exhaustive`: gli stati intermedi prodotti indipendentemente dai diversi worker devono anche poter essere combinati.

Questa proprietà è rappresentata da `Combinable[B]`, che definisce l'operazione di combinazione tra due stati intermedi. Come mostrato in Figura 5.2, `ParallelCollector` combina le due proprietà richieste, estendendo un `Collector` con politica `Exhaustive` e `Combinable[B]`.

Il type system garantisce quindi che una pipeline parallela possa utilizzare soltanto collector che richiedono l'elaborazione completa dello stream e i cui risultati parziali siano combinabili.

## Collector specializzati

Alcune operazioni terminali particolarmente comuni, come `toArray` e `summing`, dispongono di un trattamento specializzato da parte della libreria. Come mostrato in Figura 5.2, sono rappresentate rispettivamente dai tipi opachi `ToArrayCollector` e `SummingCollector`, entrambi sottotipi di `ParallelCollector`.

Questi collector non vengono utilizzati come normali implementazioni runtime di `Collector`: la loro presenza nella pipeline viene riconosciuta dal compilatore e determina la selezione di una strategia di code generation dedicata. L'uso di tipi opachi permette di esporre staticamente le proprietà necessarie, come la compatibilità con il parallelismo, senza rendere visibili all'utente i dettagli della loro implementazione.

Collector predefiniti come `toList` e `toSet` non ricevono invece un trattamento speciale e si comportano come normali collector definiti attraverso l'interfaccia pubblica.

### 5.1.2 Creazione dello stream: `from`

La costruzione di una pipeline avviene a partire dal metodo `from`. Oltre a identificare la sorgente degli elementi, il tipo restituito da `from` determina quali modalità di esecuzione (parallela/sequenziale) possono essere richieste successivamente.

Non tutte le sorgenti possiedono infatti le caratteristiche necessarie per essere sfruttate parallelamente. L'API rappresenta questa proprietà attraverso il tipo restituito: una sorgente per la quale è disponibile l'esecuzione parallela viene rappresentata come un `ParallelizableSource[A]` (Figura 5.1).

Il metodo `parallel` non appartiene quindi genericamente a ogni `Stream`, ma solamente alle sorgenti per le quali la libreria dichiara esplicitamente il supporto alla parallelizzazione. Una pipeline costruita a partire da una sorgente non parallelizzabile non può essere convertita in una `ParallelStream`, l'operazione semplicemente non è disponibile nel tipo statico restituito all'utente.

### 5.1.3 Tipologie di stream

La modalità di esecuzione della pipeline è rappresentata staticamente attraverso la gerarchia di tipi mostrata in Figura 5.1. `SequentialStream` e `ParallelStream` distinguono rispettiva-



mente le pipeline sequenziali e parallele, mentre `ParallelizableSource` identifica le sorgenti sequenziali per le quali è disponibile la transizione alla modalità parallela.

I sottotipi preservano la modalità di esecuzione lungo la pipeline: ad esempio, una `map` applicata a una `ParallelStream[A]` produce una `ParallelStream[B]`. Analogamente, la disponibilità di alcune operazioni è vincolata dal tipo statico: `skip` e `limit`, definite esclusivamente per `SequentialStream`, non sono disponibili su una `ParallelStream`.

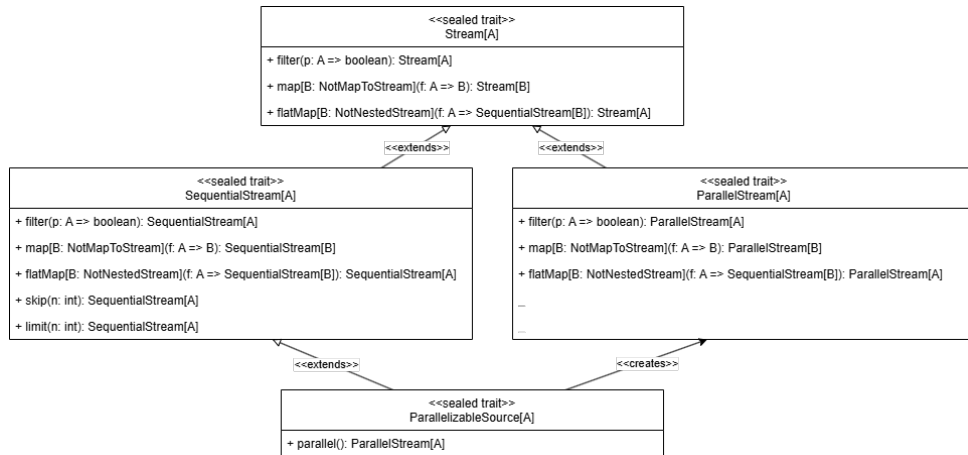


Figura 5.1: Gerarchia dei tipi utilizzati per rappresentare le modalità di esecuzione degli stream.

### 5.1.4 Operazione terminale collect

L'operazione `collect`, definita tramite extension methods, costituisce il punto nel quale la descrizione della pipeline viene trasformata nel codice che ne realizza l'esecuzione.

Una pipeline sequenziale può utilizzare genericamente un `Collector`, mentre una pipeline parallela richiede un `ParallelCollector`, cioè un collector esaustivo i cui risultati intermedi siano combinabili. Questa distinzione permette al type system di rifiutare staticamente combinazioni non supportate.

Ad esempio:

```

1 stream.parallel().collect(Collector.findFirst) // non valido
2 stream.parallel().collect(Collector.toArray)   // valido
  
```

`findFirst` è infatti `ShortCircuiting` e non soddisfa i requisiti di `ParallelCollector`, mentre `ToArrayCollector[T]` ne è un sottotipo. La gerarchia completa dei collector e delle relative politiche di terminazione è mostrata in Figura 5.2.

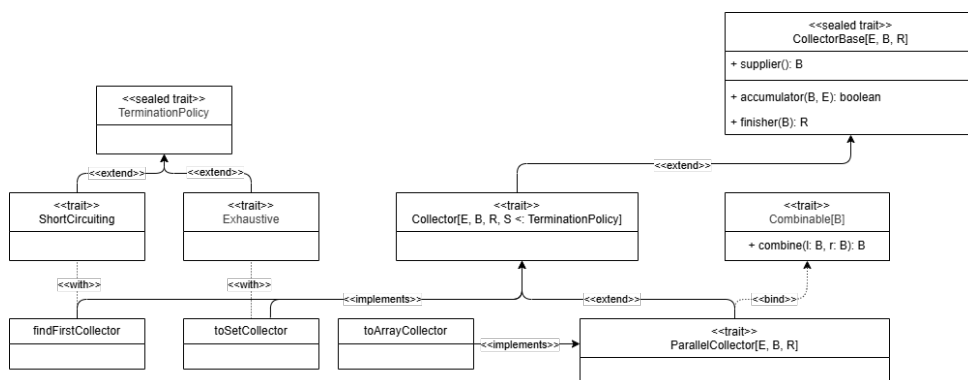


Figura 5.2: Gerarchia dei Collector e relative politiche di terminazione.

### 5.1.5 Configurazione compile-time e runtime

La libreria distingue tra due tipologie di configurazione: `CompileConfig`, relativa alle scelte che influenzano la generazione del codice, e `RuntimeConfig`, relativa ai parametri utilizzati durante l'esecuzione.

Entrambe le configurazioni sono gestite tramite il meccanismo dei `given` di Scala. La libreria fornisce un comportamento di default, permettendo quindi di utilizzare `FusedStream` senza alcuna configurazione esplicita nei casi più comuni.

Quando necessario, l'utente può comunque sovrascrivere i valori predefiniti fornendo una propria istanza `given` della configurazione desiderata nel contesto della pipeline.

In questo modo l'API mantiene una configurazione implicita e poco invasiva, lasciando comunque la possibilità di personalizzare sia il comportamento in fase di compilazione sia quello in fase di esecuzione.

### 5.1.6 Import semplificato dell'API

Gli elementi dell'API pubblica vengono riesposti direttamente attraverso l'oggetto `FusedStream`. Questo permette di importare l'intera API necessaria con una singola istruzione:

```
1 import FusedStream.*
```

ottenendo così accesso sia alle funzioni per la costruzione delle pipeline sia ai collector terminali pronti all'uso. L'obiettivo è mantenere l'interfaccia compatta e immediata, evitando che l'utente debba conoscere o importare esplicitamente i diversi oggetti interni che compongono l'API.

## 5.2 Design della Stream IR

L'obiettivo della *Stream Intermediate Representation* (Stream IR) è rappresentare la struttura logica di una pipeline e, permettere alle successive fasi del compilatore di associare all'albero nuove informazioni.

Il design della Stream IR è una struttura ad albero estensibile che prende ispirazione dal paper *Trees That Grow* [5].

Il problema principale da gestire è che un Abstract Syntax Tree non rimane immutato durante l'intero processo di compilazione. Al contrario, le fasi del compilatore introducono informazioni da associare ai nodi dell'albero. Una possibile soluzione consiste nel definire una rappresentazione differente per ciascuna fase. Questo porterebbe però duplicazione di una parte considerevole della struttura dell'AST. Un'altra opzione potrebbe essere avere un nodo unico per ogni fase con campi opzionali o un nodo base con varie estensioni per reappresentare le proprie fasi, questi approcci hanno però un problema: rendono più complicata e soggetta a possibili errori la gestione di questi dati.

E' stata invece creata una struttura parametrizzata a livello di tipo  $P$  per determinare fase alla quale appartiene l'albero.

```
1 sealed trait Phase
2 object Phase {
3   sealed trait Raw extends Phase
4   sealed trait Enriched extends Phase
5 }
6 enum StreamTree[P <: Phase, A] {
7   ...
8 }
```

$A$  Rappresenta il tipo degli elementi prodotti dal nodo.

$P$  Identifica la fase del ciclo di vita (**Phase**) nella quale il nodo si trova.

L'utilizzo della fase come parametro di tipo, rende esplicito al type system quali rappresentazioni possono essere manipolate, ne derivano benefici come pattern matching esaustivo e garanzie a compile time.

### 5.2.1 Struttura dell'albero

La Stream IR modella una pipeline come un albero i cui nodi corrispondono alle operazioni del dominio degli stream. Le sorgenti costituiscono la foglia dell'albero.

Una rappresentazione semplificata della gerarchia è mostrata in Figura 5.3.

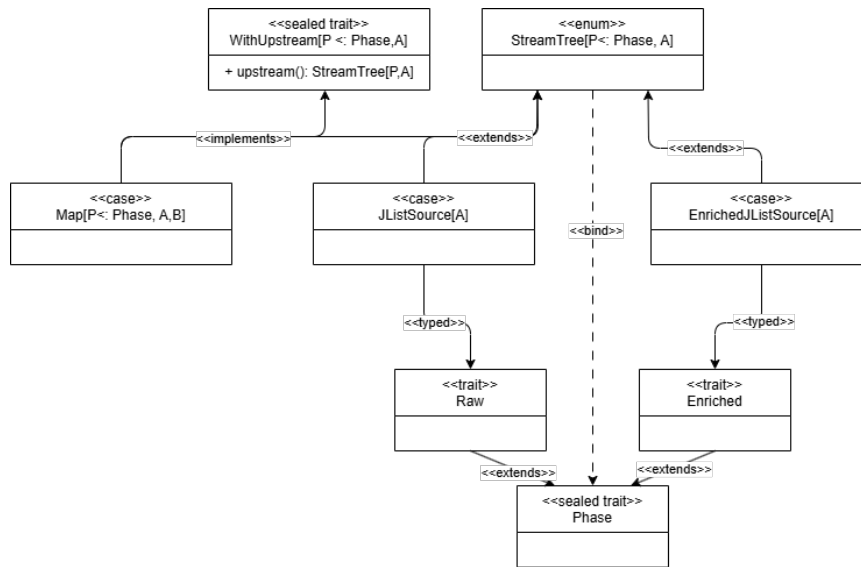


Figura 5.3: Struttura semplificata della Stream IR.

Gli operatori che non modificano la propria rappresentazione tra le due fasi sono parametrizzati direttamente tramite  $P$ , Ad esempio Map:

```
1 case Map[P <: Phase, A, B](upstream: StreamTree[P, A], ...) extends StreamTree[P, B]
```

Si noti come la fase del nodo impone anche la fase del relativo *upstream* garantendo la correttezza dell'intera struttura dati

Gli operatori la cui struttura viene invece effettivamente estesa hanno varianti specifiche per fase. Un esempio dedicato al `flatMap` è riportato nella sezione a seguire.

## 5.3 Rappresentazione per la Code Generation (OP IR)

La Stream IR descritta nella sezione precedente mantiene una rappresentazione di alto livello della pipeline e dei suoi operatori. Tale struttura è particolarmente adatta alle analisi specifiche del dominio degli stream, ma non costituisce una rappresentazione conveniente per la generazione del codice.

Per questo si introduce una seconda rappresentazione intermedia, *Operation IR* (OP IR). La traduzione verso questa rappresentazione costituisce un processo di *lowering*: operatori di alto livello vengono eliminati e sostituiti da costrutti che descrivono esplicitamente il modello di esecuzione della pipeline. A differenza della Stream IR, la OP IR non modella quindi il dominio degli stream.

La rappresentazione è organizzata attorno a due categorie fondamentali: *Values*, che descrivono espressioni che producono un valore, e *Operations*, che descrivono azioni e strutture di controllo. Un programma nella OP IR è una sequenza di operazioni e un valore che ne costituisce il risultato finale.

Questo ulteriore linguaggio intermedio permette di suddividere la generazione del codice in due fasi con responsabilità distinte. La prima opera sulla rappresentazione logica della pipeline e, sulla base degli operatori presenti, della sorgente, dell'operazione terminale e delle proprietà ricavate durante l'analisi, come la cardinalità e l'allineamento degli indici, determina la strategia di esecuzione più adatta e la esprime attraverso la OP IR. La seconda fase è invece meccanica: opera su un insieme ristretto di primitive della OP IR e ne effettua il lowering nei corrispondenti costrutti Scala. Le decisioni relative a come implementare la pipeline sono quindi prese prima della generazione effettiva del codice.

### 5.3.1 Valori

I *Values* rappresentano espressioni che producono un risultato e sono utilizzati come operandi dalle operazioni della OP IR. La gerarchia è composta da innanzitutto valori elementari, come costanti o riferimenti a Simboli e da valori composti che descrivono operazioni su altri valori come confronti, operazioni aritmetiche o applicazioni di funzioni, sono inoltre presenti valori specifici come accessi a array o iteratori.

Un **Value** descrive quindi un'espressione che può essere incorporata all'interno di una **Op**. Ad esempio, la condizione di un ciclo **While** è rappresentata come un **Value[Boolean]**, mentre il valore assegnato da una **AssignVal** è a sua volta rappresentato tramite un **Value**. Un esempio parziale della gerarchia è riportato in Figura 5.4.

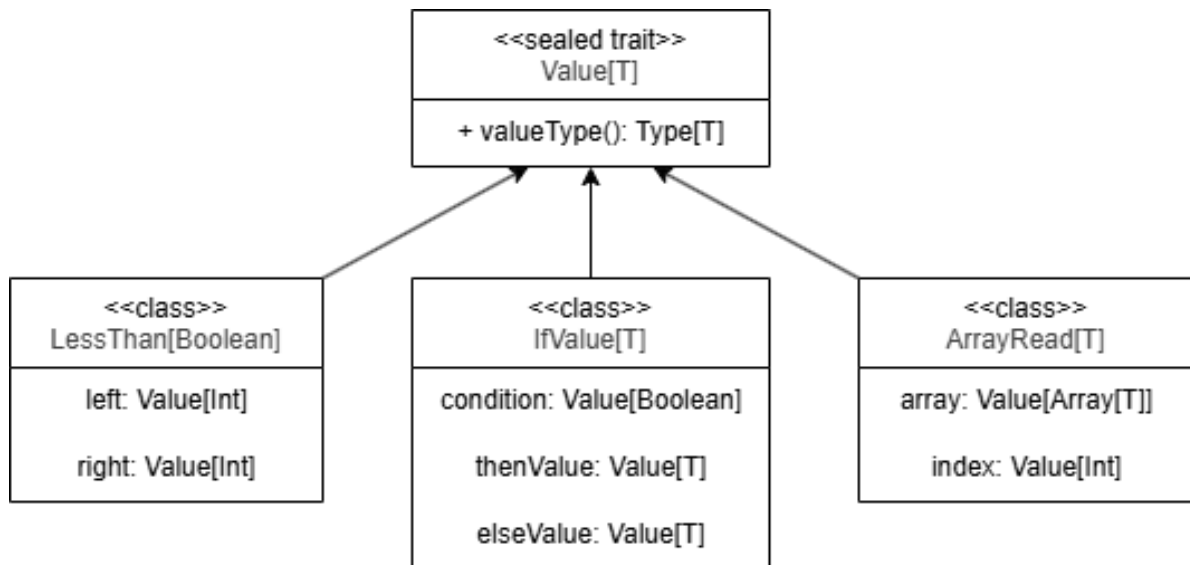


Figura 5.4: Gerarchia e rappresentazione dei valori nella OP IR.

Ogni valore possiede l'informazione relativa al tipo che rappresentano, permette quindi di esprimere staticamente i vincoli, a livello di tipi tra le diverse primitive della IR. Per esempio in **LessThan**, entrambi gli operandi devono essere valori interi, mentre il risultato è necessariamente un **Value[Boolean]**.

La sola informazione contenuta nel parametro generico non è però sufficiente per la successiva generazione del codice, ogni **Value[T]** espone quindi anche un **Type[T]**, attraverso il campo **valueType**.

Si realizza così una forma di *type capture*: oltre a conoscere staticamente che un nodo produce un valore di tipo T, la IR conserva una rappresentazione del tipo utilizzabile dalle Scala Quotes.

Per tipi noti in maniera statica, per valori parametrici viene invece propagata attraverso un parametro contestuale `Type[T]`.

Come approfondito nella Sezione 6.4.1, questa informazione viene poi sfruttata durante il lowering per costruire espressioni Scala correttamente tipizzate.

### 5.3.2 Operazioni

Le *Operations* rappresentano invece le azioni che costituiscono il corpo del programma generato.

La OP IR comprende operazioni elementari per la dichiarazione e l'aggiornamento dello stato, come `Declare`, `AssignVal` o `Inc`. Il controllo del flusso è espresso esplicitamente attraverso `If` e `While`.

Sono inoltre presenti operazioni specializzate per le strategie di materializzazione utilizzate dalla libreria. `ArrayWrite`, ad esempio, rappresenta direttamente la scrittura di un elemento in un array, mentre `DynamicArrayBuilderAdd` rappresenta l'inserimento in un buffer la cui dimensione non è nota staticamente.

## 5.4 La collection strategy

La fase terminale di una stream è rappresentata internamente attraverso una `CollectionStrategy` (Figura 5.5). Questa astrazione descrive la strategia con cui gli elementi prodotti dalla pipeline devono essere consumati e aggregati per ottenere il risultato finale.

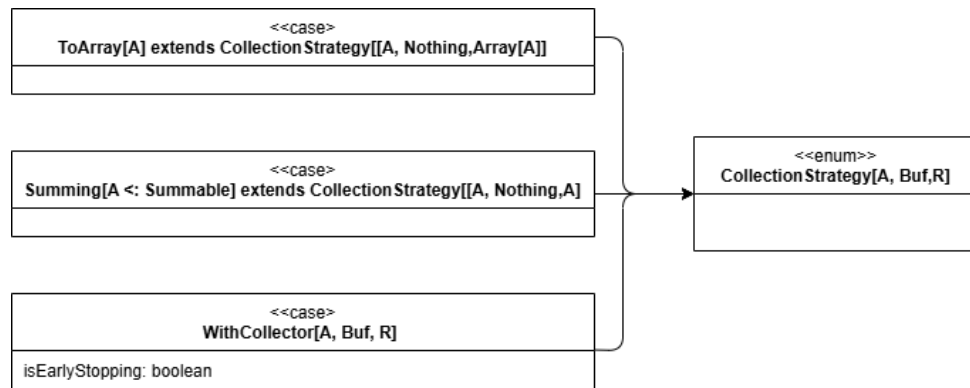


Figura 5.5: Diagramma UML delle classi per la gerarchia `CollectionStrategy`.

L'introduzione di questa rappresentazione permette di separare il collector espresso dall'utente nella DSL dalla modalità con cui il compilatore genera il codice relativo all'operazione terminale. In particolare, i collector possono essere suddivisi in due categorie.

La prima comprende i *collector concreti*, che forniscono direttamente l'implementazione delle operazioni necessarie alla raccolta. Tali operazioni vengono quindi invocate dal codice generato durante l'esecuzione della stream.

La seconda comprende invece i *collector opachi*. Questi non costituiscono l'implementazione effettivamente utilizzata a runtime, ma agiscono come placeholder tipati all'interno della DSL. Il compilatore li riconosce durante il parsing e li sostituisce con una `CollectionStrategy` specializzata, la cui implementazione è definita direttamente all'interno della libreria. Appartengono a questa categoria, ad esempio, i collector utilizzati per le operazioni `toArray` e `summing`.

Questa distinzione consente quindi di specializzare le operazioni terminali note alla libreria, evitando di rappresentarle necessariamente attraverso il protocollo generale di un collector.

# Capitolo 6

## Implementazione

### 6.1 Parsing

La fase di parsing ha l'obiettivo di trasformare l'albero sintattico tipizzato prodotto dal compilatore Scala in una `StreamIR` (Sezione 5.2) e di estrarre, a partire dal collector (Figura 5.2), la corrispondente `CollectionStrategy`. Il parsing della pipeline e quello dell'operazione terminale sono mantenuti separati e vengono successivamente combinati nella rappresentazione utilizzata dalle fasi successive.

#### 6.1.1 Parsing della stream

Il parsing della stream trasforma l'AST tipizzato ricevuto dalla macro nella Stream IR descritta nella Sezione 5.2. Il parser riconosce le operazioni della DSL all'interno dei nodi dell'AST Scala e ricostruisce ricorsivamente la struttura logica della pipeline. Un esempio completo della trasformazione dall'AST Scala alla corrispondente Stream IR è riportato nella Sezione 6.5.

#### Rappresentazione del risultato del parsing

Il risultato intermedio è rappresentato da `ParsedTree`, che consente al parser di restituire un albero senza che il chiamante debba conoscerne a priori il tipo di output. Quest'ultimo viene infatti determinato durante il parsing ed è rappresentato mediante l'abstract type member `Out`.

L'implementazione mantiene contemporaneamente `current: StreamTree[Phase.Raw, Out]` e `outType: Type[Out]`. Il primo conserva la relazione statica tra l'albero e il tipo degli elementi prodotti, mentre il secondo ne mantiene l'evidenza richiesta dalle API di `scala.quoted`.

Il tipo può essere recuperato nelle successive operazioni attraverso un path-dependent type:

```
1 type In = upstream.Out
2 given Type[In] = upstream.outType
```

Il parser può quindi tipizzare correttamente funzioni come `In => Boolean` per `filter` o `In => Out` per `map`.

#### Riconoscimento delle operazioni

Le operazioni della DSL vengono riconosciute mediante pattern matching sulla struttura dell'AST. In forma semplificata:

Listing 6.1: Riconoscimento delle operazioni della DSL

```
1 expr match
2   case Inlined(_, _, inner) => parseTerm(inner)
3   case Apply(TypeApply(NamedMethod("from"), _), List(source)) => ...
```

```

4 case Apply(Select(upstream, "filter"), List(predicate)) => ...
5 case Apply(Select(upstream, "skip"), List(n)) => ...
6 case Apply(Select(upstream, "limit"), List(n)) => ...
7 case Apply(TypeApply(Select(upstream, "map"), _, List(mapper)) => ...
8 case Apply(TypeApply(Select(upstream, "flatMap"), _, List(mapper)) => ...
9 case Apply(Select(upstream, "parallel"), _) => parseTerm(upstream)
10 case Block(statements, expr) => ...

```

Ogni caso analizza ricorsivamente l'upstream e costruisce il corrispondente nodo della Stream IR. Il tipo di output dell'upstream viene recuperato attraverso `upstream.Out` e la relativa evidenza `upstream.outType`, mantenendo tipizzate sia le funzioni ricevute dalla pipeline sia i nodi della IR.

### Parsing ricorsivo di flatMap

`flatMap` richiede un trattamento specifico poiché la funzione ricevuta produce a sua volta una `Stream`, che deve essere sottoposta ricorsivamente allo stesso processo di parsing. Si consideri:

```

1 FusedStream
2   .from(numbers)
3   .flatMap(x =>
4     FusedStream.from(elements(x)).filter(y => y > 0)
5   )

```

Il parametro `x` rappresenta un elemento che sarà disponibile soltanto a runtime. Per poter analizzare staticamente la stream interna viene quindi creato un binder tipizzato che ne rappresenta simbolicamente il valore:

```

1 type In = upstream.Out
2 given Type[In] = upstream.outType
3
4 val flatMapBinder = createConstant[In]("flatMapElem")
5 val elemRef = Ref(flatMapBinder).asExprOf[In]

```

La funzione viene interpretata come `In => Stream[Out]` e applicata al binder mediante  $\beta$ -reduction:

```

1 val innerStreamExpr =
2   Expr.betaReduce('{ $function($elemRef) })
3
4 val innerTree = parseTerm(innerStreamExpr.asTerm)

```

Si ottengono così due sottoalberi: l'upstream esterno e la stream interna prodotta da `flatMap`. Il nodo della IR conserva inoltre il simbolo del binder che li collega.

Durante la successiva code generation il binder verrà associato all'elemento corrente prodotto dall'upstream. La stream interna può quindi essere analizzata e generata a compile time pur dipendendo da un valore disponibile soltanto a runtime.

#### 6.1.2 Parsing del collector

L'obiettivo di questa operazione è ricondursi a una *collection strategy* (approfondita nella Sezione 5.4) a partire dalla rappresentazione ricevuta. Il primo passo consiste nell'analizzarne il tipo associato all'espressione confrontandolo con i collector opachi della libreria, se è questo il caso il parser costruisce direttamente la strategia specializzata corrispondente. Nel caso generale, invece, il collector fornito dall'utente deve essere preservato, viene quindi estratta la *stop policy*

per determinare staticamente se il collector supporti lo short-circuiting. Il risultato di questa analisi viene infine incorporato nella strategia generica `WithCollector`. La trasformazione può essere rappresentata come:

$$\begin{aligned} \text{ToArrayCollector}[A] &\longrightarrow \text{ToArray}[A] \\ \text{SummingCollector}[A] &\longrightarrow \text{Summing}[A] \\ \text{CollectorBase}[A, \text{Buf}, R] &\longrightarrow \text{WithCollector}[A, \text{Buf}, R] \end{aligned}$$

Nota la tipizzazione su `TerminationPolicy` a questo punto viene persa, è infatti utilizzata solo per definire la grammatica della dsl e viene convertita nel valore booleano `isEarlyStopping` dentro `WithCollector`

### 6.1.3 Parsing della configurazione statica

La libreria effettua un'ulteriore operazione di parsing relativa alla configurazione di compilazione. A differenza del parsing della stream, basato sull'analisi del TASTy, in questo caso l'estrazione viene effettuata mediante quoted pattern matching. Poiché la configurazione deve essere interamente nota a compile time, i valori dei suoi campi devono poter essere estratti staticamente dall'espressione. Sono quindi accettate istanziazioni di `CompileConfig` i cui argomenti siano costanti note in fase di compilazione, come mostrato dal seguente `FromExpr`:

Listing 6.2: Unapplier con quoted pattern matching per configurazione statica

```

1 given FromExpr[CompileConfig] with {
2   def unapply(expr: Expr[CompileConfig])(using Quotes): Option[CompileConfig] = {
3     expr match {
4       // CompileConfig(unsafe, logging)
5       case '{ CompileConfig($u, $l) } =>
6         for {
7           unsafe <- u.value
8           logging <- l.value
9         } yield CompileConfig(unsafe, logging)
10
11      // new CompileConfig(unsafe, logging)
12      case '{ new CompileConfig($u, $l) } =>
13        for {
14          unsafe <- u.value
15          logging <- l.value
16        } yield CompileConfig(unsafe, logging)
17    }
18  }
19 }
```

## 6.2 Enrichment

La rappresentazione prodotta dal parser descrive la struttura logica della pipeline, ma non contiene ancora tutte le informazioni necessarie alla generazione del codice. La fase di *enrichment* completa quindi la `StreamTree` con informazioni determinate staticamente e utilizzate nelle successive fasi di generazione.

L'enrichment svolge tre attività principali: materializza le espressioni che devono essere valutate una sola volta, introduce e propaga lo stato e le condizioni necessarie al controllo dell'esecuzione e determina proprietà globali della pipeline, quali la cardinalità del risultato e l'allineamento degli indici.



### 6.2.1 Materializzazione delle espressioni

Consiste nell'estrarre le espressioni la cui valutazione deve avvenire una sola volta prima dell'esecuzione della pipeline. Per esempio i parametri delle operazioni `skip` e `limit` quando presenti, vengono materializzati in variabili immutabili. In questo modo un'espressione come:

```
1 stream.limit(computeLimit())
```

non provoca la rivalutazione di `computeLimit()` a ogni iterazione.

Lo stesso principio viene applicato alle funzioni utilizzate da operatori come `map` e `filter`.

### 6.2.2 Stato e condizioni di terminazione

Durante l'enrichment vengono introdotte le informazioni di stato necessarie all'esecuzione degli operatori. Le operazioni `skip` e `limit` richiedono ad esempio un contatore che rappresenti la posizione corrente nel punto della pipeline in cui sono applicate.

Nel caso di `limit`, lo stato ha anche effetto sulla terminazione della pipeline: raggiunto il numero massimo di elementi non è più necessario interrogare la sorgente. La condizione

$$\text{sliceCounter} < \text{until}$$

viene quindi aggiunta all'insieme dei predicati di terminazione e propagata verso i nodi a monte.

Lo stesso meccanismo viene utilizzato per i collector *short-circuiting*. Quando la strategia di raccolta consente la terminazione anticipata, viene introdotta una variabile mutabile che rappresenta la possibilità di continuare l'elaborazione. Quando il collector determina che il risultato è completo, tale variabile viene invalidata, causando l'arresto dei cicli interessati.

La condizione di un ciclo generato non è quindi costituita solamente dalla condizione della sorgente, ma dalla sua congiunzione con tutti i predicati propagati:

$$C_{\text{loop}} = C_{\text{source}} \wedge p_1 \wedge p_2 \wedge \dots \wedge p_n.$$

`FlatMap` rappresenta un caso particolare: il relativo nodo contiene sia l'upstream della pipeline esterna sia la stream interna prodotta dalla funzione. Quest'ultima viene arricchita a sua volta e le informazioni ottenute vengono associate al nodo.

L'enrichment determina pertanto il livello di annidamento al quale stato e condizioni dovranno essere collocati durante la successiva generazione del codice: lo stato appartenente alla stream interna deve essere infatti inizializzato nuovamente per ciascun elemento prodotto dalla stream esterna. Ad esempio, il contatore associato a un `limit` interno alla `flatMap` non può essere dichiarato globalmente, ma deve essere ricreato all'interno della corrispondente iterazione. Viceversa, una condizione di terminazione introdotta a valle della `flatMap` deve poter arrestare sia l'iterazione interna sia quella esterna.

### 6.2.3 Analisi della cardinalità

Terminato l'enrichment dell'albero vengono determinate due proprietà utilizzate per selezionare la strategia di generazione: la cardinalità del risultato e l'allineamento degli indici.

La cardinalità è rappresentata mediante:

```
1 enum Cardinality {  
2   case Exact(size: Expr[Int])  
3   case UpperBound(size: Expr[Int])  
4   case Unknown  
5 }
```

Una sorgente indicizzabile, come un array o una `java.util.List`, possiede cardinalità esatta. Operazioni come `map` la preservano, mentre `filter`, potendo eliminare elementi ma non aumentarne il numero, la trasformano in un limite superiore. Sorgenti genericamente iterabili e operazioni come `flatMap`, per le quali non è possibile determinare staticamente il numero di elementi prodotti, conducono invece a una cardinalità sconosciuta.

A questa informazione si affianca `hasAlignedIndexes`, che indica se l'indice dell'elemento nella sorgente può essere utilizzato direttamente come indice del risultato. La proprietà è inizialmente valida per sorgenti indicizzabili ed è preservata da operazioni come `map` e `limit`; viene invece invalidata da `filter`, `skip` e `flatMap`, che rompono la corrispondenza tra posizione letta e posizione prodotta.

## 6.3 Traduzione in OP IR

Una volta completato l'enrichment, la Stream IR contiene le informazioni necessarie per determinare come la pipeline debba essere generata. La fase successiva traduce quindi la rappresentazione logica della stream nella *Operation IR* (OP IR), introdotta nella Sezione 5.3.

È in questa fase che vengono prese le principali decisioni relative al piano di esecuzione. La strategia generata dipende dall'operazione terminale, dalla modalità sequenziale o parallela, dalla cardinalità della pipeline, dall'allineamento degli indici e dalle caratteristiche della sorgente. Il risultato è un `Program[T]` composto da operazioni e da un valore finale, nel quale i concetti propri del dominio degli stream sono stati tradotti in dichiarazioni, condizioni, cicli e operazioni sui valori.

La fase di lowering descritta nella Sezione 6.4 può quindi limitarsi alla traduzione meccanica di tali operazioni verso l'AST Scala.

### 6.3.1 Selezione della strategia terminale

Il primo livello decisionale riguarda l'operazione terminale. La `CollectionStrategy` ottenuta durante il parsing distingue tre casi:

- `ToArray`, per il quale viene selezionata una strategia di materializzazione dipendente dalla cardinalità della pipeline;
- `Summing`, tradotto in un accumulatore numerico specializzato sul tipo degli elementi;
- `WithCollector`, che utilizza il protocollo generale `supplier/accumulator/finisher`.

### 6.3.2 Materializzazione del risultato

Il terminale `toArray` mostra come le informazioni determinate durante l'enrichment influenzino direttamente il piano generato. La strategia dipende principalmente dalla cardinalità e dall'allineamento degli indici. Con cardinalità esatta e indici allineati, come nel caso di una pipeline composta esclusivamente da `map` su una sorgente indicizzabile, è possibile preallocare l'array finale e utilizzare direttamente l'indice della sorgente.

La stessa proprietà è particolarmente utile in parallelo: poiché la posizione finale di ogni elemento è nota indipendentemente, i worker possono scrivere in regioni distinte dello stesso array mediante `ParallelCombine.ArrayDirect()`, senza produrre risultati intermedi.

Quando è noto solamente un limite superiore, ad esempio in presenza di `filter`, viene comunque preallocato un array, ma viene mantenuto un indice di output separato. Al termine l'array viene eventualmente troncato al numero di elementi effettivamente prodotti.

Se invece la cardinalità è sconosciuta, come per una sorgente genericamente iterabile o per `flatMap`, viene utilizzato un buffer dinamico e l'array viene materializzato solamente al termine dell'elaborazione.

Nel caso parallelo gli stessi principi vengono applicati a ciascun chunk: quando non è possibile scrivere direttamente nell'array finale, ogni partizione produce un risultato locale che viene successivamente combinato.

### 6.3.3 Generazione della pipeline e delle sorgenti

Determinata la strategia terminale, viene generato il corpo della pipeline. La generazione segue un modello basato su una funzione di emissione:

```
1 private type Emit[A] = Emitted[A] => Op
2
3 private final case class Emitted[A](
4   elem: Value[A],
5   sourceIndex: Option[Value[Int]]
6 )
```

La funzione `Emit` rappresenta il comportamento del downstream quando il nodo corrente produce un elemento. Gli operatori intermedi non generano quindi un ciclo autonomo, ma trasformano questa continuation e delegano la generazione al proprio upstream. Il ciclo viene emesso solamente quando la ricorsione raggiunge la sorgente.

Ad esempio, `filter` condiziona l'invocazione del downstream:

$$E'(x) = \text{if } p(x) \text{ then } E(x),$$

che nella OP IR corrisponde a:

```
1 If(
2   ApplyFun(filter.predicate, emitted.elem),
3   emit(emitted)
4 )
```

Una `map` modifica invece il valore propagato:

```
1 Declare(mapped, ApplyFun(map.function, emitted.elem))
2 emit(Emitted(SymbolRef(mapped), emitted.sourceIndex))
```

Il caso più articolato è `flatMap`. Il nodo contiene infatti una seconda Stream IR che rappresenta la pipeline interna. Il binder creato durante il parsing viene associato all'elemento corrente dell'upstream:

```
1 Declare(flatMap.elemSymbol, emitted.elem)
```

La stream interna viene quindi tradotta ricorsivamente utilizzando la stessa continuation del `flatMap`. Il risultato è l'annidamento diretto del ciclo interno nel ciclo esterno, senza materializzare una stream intermedia. Concettualmente:

```
1 while (outerCondition) {
2   val binder = nextOuter()
3   while (innerCondition) {
4     val y = nextInner(binder)
5     if (p(y))
6       emit(y)
7   }
8 }
```

La ricorsione termina infine quando viene raggiunta la sorgente, che introduce il ciclo effettivo. La tipologia concreta della sorgente può determinare ulteriori specializzazioni. Per una `java.util.List`, ad esempio, viene distinto il caso `ArrayList`, per il quale l'accesso indicizzato evita la creazione dell'iteratore. Se abilitato dalla configurazione di compilazione, è inoltre possibile accedere direttamente all'array interno tramite `VarHandle`.

### 6.3.4 Generazione del piano parallelo

Nel caso parallelo, la OP IR raccoglie esplicitamente le informazioni necessarie a descrivere la partizione del lavoro:

- l'intervallo della sorgente interessato dalla computazione;
- la sequenza di operazioni da eseguire su ciascuna partizione;
- l'eventuale risultato locale prodotto da ogni partizione;
- la strategia di combinazione dei risultati parziali;
- il riferimento al risultato finale del calcolo.

La OP IR non descrive quindi i dettagli del meccanismo di scheduling: rappresenta il parallelismo come operazione di alto livello, lasciando alla successiva fase di lowering la sua traduzione nelle primitive runtime.

## 6.4 Lowering

La fase di lowering costituisce l'ultimo passaggio del compilatore interno di FusedStream. Dato un `Program[T]`, produce un'espressione Scala `Expr[T]`, che viene restituita al compilatore e reinserita nel normale processo di compilazione.

A questo punto le decisioni relative alla strategia di esecuzione sono già state prese: il lowering effettua una traduzione dell'OP IR nei corrispondenti nodi dell'AST Scala. La trasformazione è implementata principalmente attraverso due funzioni: `lowerOp`, per le operazioni con effetti o controllo del flusso, e `lowerValue`, per le espressioni che producono un valore.

### 6.4.1 Traduzione dell'OP IR

La traduzione viene effettuata mediante pattern matching sui nodi dell'OP IR, le operazioni vengono ricondotte ai corrispondenti costrutti dell'AST Scala. Analogamente, i nodi `Value[T]` vengono tradotti ricorsivamente in `Expr[T]`. Riferimenti, costanti ed espressioni Scala già disponibili possono essere convertiti direttamente, mentre gli operatori composti richiedono il lowering ricorsivo dei propri operandi. Ad esempio:

```
1 LessThan(left, right)
```

viene tradotto in:

```
1 '{
2   ${lowerValue(left)} < ${lowerValue(right)}
3 }
```

Lo stesso procedimento viene applicato alle operazioni logiche, confronti, accessi ad array e alle operazioni sui collector, eliminando progressivamente l'OP IR fino a ottenere esclusivamente nodi appartenenti all'AST Scala.

Un aspetto rilevante di questa trasformazione è la conservazione delle informazioni di tipo. Per costruire un `Expr[T]`, scala richiede che sia disponibile un'evidenza `Type[T]`. Per questo motivo, come descritto nella Sezione 5.3, ciascun `Value[T]` conserva, oltre al parametro di tipo, anche il corrispondente `valueType`.

Durante il pattern matching sull'OP IR tale informazione viene recuperata e resa disponibile localmente. Ad esempio, nel lowering di una scrittura su array:

```
1 case write: ArrayWrite[t] =>
2   given Type[t] = write.valueType
3
4   val array = lowerValue[Array[t]](write.array)
5   val index = lowerValue[Int](write.index)
6   val value = lowerValue[t](write.value)
7
8   '{ $array($index) = $value }
```

L'informazione conservata nella rappresentazione intermedia permette quindi di mantenere il collegamento tra il sistema di tipi dell'OP IR e quello delle Scala Quotes fino all'ultima fase della generazione.

### 6.4.2 Applicazione e inlining delle funzioni

Un caso particolarmente importante è costituito da `ApplyFun`. In questo caso viene utilizzata la  $\beta$ -reduction, descritta nella Sezione 3.3.3. Quando la funzione è una lambda, la  $\beta$ -reduction ne inserisce il corpo direttamente nel punto di utilizzo. Ad esempio:

```
1 ApplyFun(x => x * 2, elem)
```

viene ridotto a:

```
1 elem * 2
```

In questo modo la funzione non deve essere mantenuta come entità invocabile nel codice generato, eliminando il relativo dispatch.

### 6.4.3 Lowering del parallelismo

Il nodo `Parallel` costituisce il caso più articolato del lowering. La fase precedente ha già determinato la strategia di esecuzione e la modalità di combinazione dei risultati; il lowering deve quindi trasformare tale piano nel corrispondente codice concorrente.

Il tipo di `ParallelCombine` determina l'implementazione specializzata da generare. La libreria prevede tre strategie specializzate e una generica:

- **Sum**: combina i risultati parziali mediante somma;
- **ArrayConcat**: concatena gli array prodotti dai singoli chunk;
- **ArrayDirect**: scrive direttamente nell'array finale condiviso;
- **GenericCombiner**: combina i risultati tramite il `Combinable` associato a un `Collector` generico.

L'intervallo della sorgente viene partizionato in chunk. Quando la strategia richiede risultati intermedi, questi vengono memorizzati separatamente per ciascun chunk e successivamente combinati. Nel caso della somma, ad esempio:

```
1 partials(chunkIdx) = localResult
```

Al termine dell'elaborazione i risultati parziali vengono ridotti sequenzialmente per ottenere il risultato finale.

Anche lo scheduling viene definito a runtime sulla base della dimensione della sorgente e della configurazione. Il numero di chunk è limitato dalla dimensione dell'input e determinato in funzione del numero di worker. L'assegnazione non è statica: ogni worker acquisisce un chunk e, terminata l'elaborazione, richiede dinamicamente il successivo attraverso un contatore atomico condiviso. In questo modo i chunk possono essere distribuiti progressivamente tra i worker senza fissarne preventivamente l'assegnazione e bilanciando dinamicamente il carico. Non è presente un meccanismo di work-stealing.

## 6.5 Esempio di traduzione end-to-end

Questa sezione ripercorre passo passo il processo di analisi e generazione del codice effettuato dalla libreria a partire da una semplice pipeline.

L'intero processo segue il flusso dell'applicativo:

Scala AST  $\rightarrow$  StreamIR  $\rightarrow$  Enriched StreamIR  $\rightarrow$  Execution Plan  $\rightarrow$  OpIR  $\rightarrow$  Scala AST  $\rightarrow$  JVM byte

### 6.5.1 L'input alla libreria

La pipeline utilizzata come esempio è mostrata nel Listato 6.5.1:

```
1 FusedStream.from(numbers)
2   .map(f)
3   .filter(p)
4   .toArray
```

La macro riceve l'espressione sotto forma di AST Scala tipizzato; in particolare, il parser riceve l'albero mostrato nel Listato 6.5.1:

```
1 Inlined(
2   None,
3   Nil,
4   Apply(
5     Select(
6       Apply(
7         Apply(
8           TypeApply(
9             Select(
10              Apply(
11                TypeApply(
12                  Select(Ident("FusedStream"), "from"),
13                    List(Inferred())
14                  ),
15                  List(Ident("numbers"))
16                ),
17                "map"
18              ),
19              List(Inferred())
20            ),
21            List(
```

```

22     Block(
23       List(
24         DefDef(
25           "$anonfun",
26           List(TermParamClause(List(ValDef("i", Inferred(), None)))),
27           Inferred(),
28           Some(
29             Apply(
30               Select(Ident("i"), "*"),
31               List(Literal(IntConstant(2)))
32             )
33           )
34         )
35       ),
36       Closure(Ident("$anonfun"), None)
37     )
38   ),
39   List(
40     Apply(
41       TypeApply(Ident("given_NotMapToStream_T"), List(Inferred())),
42       List(Ident("value"))
43     )
44   )
45 ),
46 "filter"
47 ),
48 List(
49   Block(
50     List(
51       DefDef(
52         "$anonfun",
53         List(TermParamClause(List(ValDef("i", Inferred(), None)))),
54         Inferred(),
55         Some(
56           Apply(
57             Select(
58               Apply(
59                 Select(Ident("i"), "%"),
60                 List(Literal(IntConstant(2)))
61               ),
62               "=="
63             ),
64             List(Literal(IntConstant(0)))
65           )
66         )
67       )
68     ),
69     Closure(Ident("$anonfun"), None)
70   )
71 )
72 )
73 )
74 )

```

E l'espressione rappresentate il terminale

```
1 fuse.FusedStream.toArray[scala.Int]
```

### 6.5.2 Il risultato del parsing

La prima fase consiste nella traduzione dell'AST Scala nella rappresentazione intermedia ad alto livello descritta nella Sezione 5.2. La Stream IR corrispondente all'AST del Listato 6.5.1 è riportata nel Listato 6.3:

Listing 6.3: Rappresentazione ottenuta al termine del parsing

```
1  Filter(  
2    upstream = Map(  
3      upstream = JListSource(  
4        source = list,  
5        elementType = Int  
6      ),  
7      function = (i: Int) => i * 2,  
8      inputType = Int,  
9      outputType = Int  
10   ),  
11   predicate = (i: Int) => i % 2 == 0,  
12   elementType = Int  
13 )
```

### 6.5.3 L'output dell'enrichment

La StreamIR del Listato 6.3 viene quindi sottoposta alla fase di enrichment (Sezione 6.2). La Enriched StreamIR risultante è illustrata nel Listato 6.4:

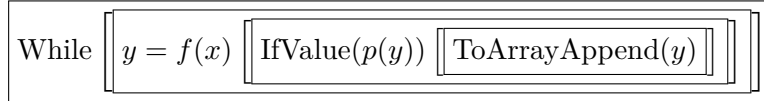
Listing 6.4: StreamIR dopo la fase di enrichment

```
1  declarations = [  
2    // Dichiarazione della variabile dove viene materializzata la fonte  
3    Declare(source$macro$1, ScalaExpr(list)),  
4    // Dichiarazione della variabile che rappresenta la size della fonte  
5    Declare(sourceSize$macro$1, ScalaExpr(source$macro$1.size()))  
6  ]  
7  
8  
9  stream =  
10  Filter(  
11    upstream = Map(  
12      upstream = EnrichedJListSource(  
13        source = ScalaExpr(source$macro$1),  
14        size = ScalaExpr(sourceSize$macro$1),  
15        elementType = Int  
16      ),  
17      function = (i: Int) => i * 2,  
18      inputType = Int,  
19      outputType = Int  
20    ),  
21    predicate = (i: Int) => i % 2 == 0,  
22    elementType = Int  
23  )  
24  
25  streamInfo =  
26  StreamProperties(  
27    // Abbiamo un filter quindi la size non e' esatta ma un upper bound  
28    size = UpperBound(ScalaExpr(sourceSize$macro$1)),  
29    hasEarlyExit = false
```



### 6.5.4 Generazione dell'OpIR

A partire dalla rappresentazione arricchita del Listato 6.4, il compilatore determina la strategia con cui attraversare la sorgente ed eseguire le operazioni della pipeline. Il piano di esecuzione assume la forma:



Il piano di esecuzione viene espresso tramite OpIR nel Listato 6.5:

Listing 6.5: OPIR generato per una pipeline con mapping, filtro e raccolta in array

```

1 statements = [
2   // Estrae la lista di origine
3   Declare(source$macro$1, ScalaExpr(numbers)),
4   // Estrae la size della lista
5   Declare(sourceSize$macro$1, ScalaExpr(source$macro$1.size())),
6   // Crea un vettore preallocato
7   Declare(vec$macro$1, ArrayDefine[scala.Int](ScalaExpr(sourceSize$macro$1))),
8   // Dichiaro il counter e inizializza a 0
9   Declare(i$macro$1, ConstantVal(0)),
10  // Genera codice ottimizzato a runtime se e' una attaylist usiamo accesso
    indicizzato (costo o(1), niente iteratori)
11  If(
12    condition = IsInstanceOf[java.util.ArrayList[]](ScalaExpr(source$macro$1)),
13    then =
14      CodeBlock(
15        Declare(i$macro$2, ConstantVal(0)),
16        While(
17          // Ciclo diretto su size, non abbiamo condizioni come limit o skip
18          condition = LessThan(SymbolRef(i$macro$2), ScalaExpr(sourceSize$macro$1)),
19          body =
20            CodeBlock(
21              // Leggiamo valore della list in una nuova variabile.
22              // Se useUnsafe = true allora JListRead legge direttamente
23              // l'array sottostante ad arraylist tramite varhandle
24              Declare(elem$macro$1,
25                JListRead(ScalaExpr(source$macro$1),
26                  SymbolRef(i$macro$2))),
27              // Map
28              CodeBlock(
29                Declare(
30                  mapped$macro$1,
31                  ApplyFun(((i: scala.Int) => i.*(2)), SymbolRef(elem$macro$1))
32                ),
33              // Filter
34              If(
35                condition = ApplyFun(
36                  ((i: scala.Int) => i.%(2).==(0)),
37                  SymbolRef(mapped$macro$1)),
38                then =
39                  // Se la condizion e' risptettata write all'indice
40                  CodeBlock(
41                    ArrayWrite(

```

```

42         SymbolRef(vec$macro$1),
43         SymbolRef(i$macro$1),
44         SymbolRef(mapped$macro$1)),
45         Inc(i$macro$1)
46     )
47 )
48 ),
49 Inc(i$macro$2)
50 )
51 )
52 ),
53 //codice non ottimizzato, per List senza accesso diretto, usa iteratore
54 else =...
55 )
56 ],
57 result = IfValue(
58     // In base alla size dell'array lo ritorno direttamente o lo tronco
59     Equal(SymbolRef(i$macro$1),ArrayLength(SymbolRef(vec$macro$1))),
60     SymbolRef(vec$macro$1),
61     ArrayTake(SymbolRef(vec$macro$1),SymbolRef(i$macro$1))
62 )

```

### 6.5.5 Lowering verso l'AST Scala

L'elenco di operazioni viene poi tradotto nell'AST scala (qui omesso vista la verbosità e complessità). Segue invece una decompilazione del bytecode java corrispondente al codice generato:

```

1  ArrayList<Integer> numbers = new ArrayList<Integer>();
2  numbers.addAll(List.of(..));
3
4  int[] nArray;
5  RuntimeConfig conf = RuntimeConfig$.MODULE$.default();
6  ArrayList<Integer> source$macro$1 = numbers;
7  int sourceSize$macro$1 = source$macro$1.size();
8  int[] vec$macro$1 = new int[sourceSize$macro$1];
9  int i$macro$1 = 0;
10 if (source$macro$1 instanceof ArrayList) {
11     ArrayList<Integer> list$proxy1 = source$macro$1;
12     Object[] raw$macro$1 = ArrayListAccessor$.MODULE$.inline$ElementDataHandle().
13     get(list$proxy1);
14     for (int i$macro$2 = 0; i$macro$2 < sourceSize$macro$1; ++i$macro$2) {
15         int elem$macro$1 = BoxesRunTime.unboxToInt(raw$macro$1[i$macro$2]);
16         int mapped$macro$1 = elem$macro$1 * 2;
17         if (mapped$macro$1 % 2 != 0) continue;
18         vec$macro$1[i$macro$1] = mapped$macro$1;
19         ++i$macro$1;
20     }
21 } else {
22     // tramite foreach, rimosso per brevità
23 }
24 if (i$macro$1 == vec$macro$1.length) {
25     nArray = vec$macro$1;
26 } else {
27     Object object = Predef$.MODULE$.genericArrayOps(vec$macro$1);
28     nArray = (int[])ArrayOps$.MODULE$.take$extension(object, i$macro$1);
29 }
30 int[] result = nArray;

```

### 6.5.6 Il bytecode JVM generato

La corretta fusione delle operazioni può essere verificata anche ispezionando direttamente il bytecode JVM generato (riportato nel Listato 6.6):

Listing 6.6: Estratto del bytecode JVM relativo al corpo della pipeline

```
1 125: iload      10
2 127: iload      5
3 129: if_icmpge   175
4
5 132: aload      8
6 134: iload      10
7 136: aaload
8 137: invokestatic BoxesRunTime.unboxToInt
9 140: istore      11
10
11 142: iload      11
12 144: iconst_2
13 145: imul
14 146: istore      12
15
16 148: iload      12
17 150: iconst_2
18 151: irem
19 152: iconst_0
20 153: if_icmpne   169
21
22 156: aload      6
23 158: iload      7
24 160: iload      12
25 162: iastore
26 163: iinc        7, 1
27
28 169: iinc        10, 1
29 172: goto        125
```

Il risultato mostra in maniera evidente l'effetto della fusione e dell'inlining. L'operazione di mapping

$$x \mapsto x \cdot 2$$

non corrisponde più all'invocazione di una funzione, ma viene tradotta direttamente nell'istruzione JVM `imul`. Analogamente, il predicato

$$x \bmod 2 = 0$$

è rappresentato dalle istruzioni `irem` e `if_icmpne`.

Nel corpo del ciclo non sono presenti invocazioni dinamiche verso le funzioni utilizzate da `map` e `filter`. Non compaiono né chiamate a `Function.apply`, né istruzioni `invokedynamic` associate alla costruzione o all'invocazione di lambda. Anche la raccolta del risultato è ridotta a operazioni primitive: `iastore` scrive direttamente il valore trasformato nell'array di output e `iinc` aggiorna il relativo indice.

## Capitolo 7

# Testing e valutazione delle prestazioni

### 7.1 Testing

Il testing di una libreria come `FusedStream` deve affrontare due problemi:

1. **Correttezza a runtime:** accertarsi che il codice generato dalle macro produca i risultati attesi, rispetti l'ordine degli elementi e preservi la semantica delle normali collezioni Scala.
2. **Verifica a compile-time:** assicurarsi che le combinazioni non valide vengano intercettate subito dal compilatore, fornendo messaggi di errore chiari ed evitando comportamenti anomali o errori oscuri a runtime.

I test sono scritti con `MUnit` e sono organizzati su più livelli separando i controlli interni da quelli sull'API pubblica:

- **Unit test interni:** verificano i singoli stadi del motore di compilazione in isolamento.
- **Test funzionali end-to-end:** compilano ed eseguono pipeline complete scritte tramite l'API pubblica, verificando il risultato dell'esecuzione. Questi test sono suddivisi in quattro suite dedicate:
  - `SequentialStreamTest`: trasformazioni sequenziali di base e operatori di slice;
  - `PipelineSemanticsTest`: ordine di valutazione delle espressioni, isolamento dei side-effect e correttezza della fusione;
  - `SourceAndCollectorTest`: sorgenti Scala e Java, ottimizzazioni con accesso diretto alla memoria e collector terminali;
  - `ParallelStreamTest`: partizionamento del carico su più worker, concorrenza e riduzioni parallele.
- **Test di compilazione:** verificano che l'uso scorretto dell'API venga bloccato a tempo di compilazione.

#### Metriche di copertura

La copertura del codice è stata misurata tramite `sbt-scoverage` sul modulo (`core`), come riassunto in Tabella 7.1. La suite raggiunge una copertura complessiva delle istruzioni del **95.05%** (1708 su 1797 istruzioni) e una copertura dei rami decisionali del **94.67%** (320 su 338 rami).

Tabella 7.1: Copertura del codice rilevata da `sbt-scoverage` sul modulo `core`.

| Package / Sottosistema              | Istruzioni         | Coverage      | Branch coverage |
|-------------------------------------|--------------------|---------------|-----------------|
| <code>fuse.internal.parallel</code> | 100.00%            | 100.00%       |                 |
| <code>fuse.internal.ir</code>       | 100.00%            | 100.00%       |                 |
| <code>fuse.internal</code>          | 95.19%             | 92.89%        |                 |
| <code>fuse (API pubblica)</code>    | 68.75%             | 83.33%        |                 |
| <b>Totale</b>                       | <b>1708 / 1797</b> | <b>95.05%</b> | <b>94.67%</b>   |

## 7.2 Valutazione delle prestazioni

### 7.2.1 Metodologia

Le misurazioni sono fatte con *Java Microbenchmark Harness* [6], su OpenJDK 64-Bit (21.0.7-LTS) x86\_64. I risultati sono espressi come tempo medio per operazione (*Average Time,  $\mu$ s/op*).

In particolare, il comportamento della libreria viene comparato con tre implementazioni alternative:

1. **Java Stream:** le stream della stdlib di Java
2. **Scala View:** le collezioni lazy della stdlib di Scala 3
3. **Loop Java manuale:** Codice java imperativo scritto a mano, rappresenta il codice imperativo tipicamente scritto

### 7.2.2 Benchmark sequenziali

Per comprendere le caratteristiche della libreria è fondamentale distinguere tra un microbenchmark sintetico e un carico applicativo reale.

#### Scenario Monomorfo

Molti benchmark che hanno come oggetto *Stream* java si limitano a valutare una singola, semplice pipeline (ad esempio `map` seguita da `sum`):

$$\text{Source} \xrightarrow{\text{map}(t \mapsto t.\text{amount})} \xrightarrow{\text{sum}()} \text{Double}$$

In questo scenario artificiale, il compilatore JIT C2 osserva un unico target (tecnicamente i call-site sono *monomorfi*) e riesce quindi ad applicare l'inlining sia alle lambda sia all'infrastruttura a oggetti di Java Stream.

Tuttavia, come emerge chiaramente dai dati in Tabella 7.2, persino in questo contesto ideale Java Stream evidenzia limiti:

- Su collezioni piccole l'inizializzazione della pipeline introduce un costo fisso che rende Java Stream da 6.7 a oltre 11 volte più lenta di **FusedStream** e del loop manuale.
- Su volumi consistenti ( $N = 100\,000$ ), nonostante l'inlining di C2, Java Stream non raggiungere la velocità di codice imperativo classico.

Tabella 7.2: Benchmark Monomorfo: tempo medio di esecuzione ( $\mu s/op$ ).

| Implementazione  | $N = 10$     | $N = 100$    | $N = 1\,000$ | $N = 100\,000$ |
|------------------|--------------|--------------|--------------|----------------|
| FusedStream      | <b>0.006</b> | <b>0.067</b> | <b>0.914</b> | <b>504.66</b>  |
| Java Manual Loop | 0.007        | 0.068        | 0.918        | 507.56         |
| Scala View       | 0.042        | 0.331        | 3.230        | 335.44         |
| Java Stream      | 0.068        | 0.451        | 4.290        | 857.71         |

### Scenario Megamorfo (Carico applicativo reale)

Questo scenario simula un carico applicativo di analisi e reportistica su un database in-memory, eseguendo 8 query distinte su transazioni.

In Java Stream e Scala View i call site interni delle interfacce funzionali diventano *megamorfici* (più di due classi concrete osservate a runtime). La *cache* collassa, disabilitando l'inlining, la JVM è costretta a eseguire dispatch virtuali ad ogni elemento.

Come riportato in Tabella 7.3, le prestazioni di **FusedStream** equivalgono a quelle del codice Java manuale lungo tutti e quattro gli ordini di grandezza analizzati.

Tabella 7.3: Benchmark Megamorfo Sequenziale: 8 query eterogenee ( $\mu s/op$ ).

| Implementazione  | $N = 10$     | $N = 100$    | $N = 1\,000$ | $N = 100\,000$ |
|------------------|--------------|--------------|--------------|----------------|
| FusedStream      | <b>0.011</b> | <b>0.089</b> | <b>1.485</b> | <b>799.50</b>  |
| Java Manual Loop | 0.011        | 0.083        | 1.436        | 758.64         |
| Scala View       | 0.065        | 0.493        | 4.815        | 655.42         |
| Java Stream      | 0.087        | 0.501        | 4.631        | 1261.43        |

### 7.2.3 Benchmark paralleli

La valutazione del runtime parallelo analizza l'efficienza nell'esecuzione delle stesse identiche 8 query usate in precedenza. Vengono messi a confronto:

- **FusedStream.parallel()**: partizionamento statico indicizzato con chunk distribuiti dinamicamente e riduzione diretta dei parziali.
- **Java parallelStream()**: parallelismo nativo di Java basato su decomposizione ricorsiva con **Splitterator**.
- **Java Manual Parallel**: chunking manuale dell'array con accesso indicizzato classico **ArrayList.get(i)** e sottomissione di task tramite **invokeAll** su **ForkJoinPool**.

I risultati sono riassunti in Tabella 7.4.

Tabella 7.4: Benchmark Megamorfo Parallelo: 8 query eterogenee ( $\mu s/op$ ).

| Implementazione      | $N = 10$ | $N = 100$ | $N = 1\,000$ | $N = 100\,000$ |
|----------------------|----------|-----------|--------------|----------------|
| FusedStream Parallel | 10.66    | 13.11     | 14.32        | <b>435.77</b>  |
| Java Manual Parallel | 11.27    | 12.72     | 13.16        | 617.78         |
| Java Parallel Stream | 3.75     | 4.90      | 8.83         | 1029.31        |

I dati evidenziano due elementi rilevanti:

1. Su collezioni grandi ( $N = 100\,000$ ), `FusedStream.parallel()` domina nettamente lo scenario multi-core, risultando 2.36 volte più veloce di Java Parallel Stream ( $1029.31\,\mu\text{s}$ ) e 1.42 volte più veloce del loop parallelo manuale.
2. Su insiemi ridotti ( $N \leq 1\,000$ ), l'esecuzione parallela è sensibilmente più lenta di quella sequenziale a causa dei costi di sincronizzazione.
3. Un aspetto interessante è la maggiore efficienza di Java Parallel Stream per dimensioni contenute ( $N \leq 1\,000$ ). Questa differenza è dovuta principalmente all'utilizzo di primitive a basso livello (`ForkJoinPool`), che riducono i context switch rispetto all'`ExecutionContext` generico sincronizzato tramite `CountDownLatch`.

#### 7.2.4 Comportamento a freddo e livelli di compilazione JIT

Nelle applicazioni moderne (funzioni serverless o microservizi), le prestazioni durante l'avvio a freddo e i primi cicli di esecuzione sono rilevanti per contenere la latenza iniziale (*cold start*).

HotSpot adotta un'architettura di compilazione a livelli (*Tiered Compilation*):

- **Tier 0 (Interprete puro):** interpretazione pura del bytecode JVM istruzione per istruzione
- **Tier 1 (C1, no profiling):** compilazione immediata con il compilatore Client C1
- **Tier 2 (C1, basic profiling):** compilazione C1 con contatori di invocazione e branch
- **Tier 3 (C1, full profiling):** compilazione C1 con profilazione
- **Tier 4 (C2, Server):** compilazione al massimo livello con inlining speculativo, escape analysis ecc.

La Tabella 7.5 riporta i tempi medi di esecuzione sui tre livelli (interpretato, C1, C2) di compilazione nello scenario Megamorfo sequenziale.

Tabella 7.5: Confronto prestazioni sui livelli JIT nello scenario Megamorfo

| Configurazione JIT                       | FusedStream    | Java Manual   | Java Stream     | Scala View      |
|------------------------------------------|----------------|---------------|-----------------|-----------------|
| <i>Taglia <math>N = 10</math>:</i>       |                |               |                 |                 |
| Tier 0 (Interprete puro)                 | <b>0.901</b>   | 1.240 (1.4×)  | 4.839 (5.4×)    | 14.519 (16.1×)  |
| Tier 1 (C1)                              | <b>0.041</b>   | 0.037         | 0.331 (8.1×)    | 0.544 (13.3×)   |
| Tier 4 (C2 completo)                     | <b>0.011</b>   | 0.011         | 0.087 (7.9×)    | 0.065 (5.9×)    |
| <i>Taglia <math>N = 100</math>:</i>      |                |               |                 |                 |
| Tier 0 (Interprete puro)                 | <b>5.561</b>   | 11.179 (2.0×) | 16.342 (2.9×)   | 47.982 (8.6×)   |
| Tier 1 (C1)                              | <b>0.371</b>   | 0.383         | 1.159 (3.1×)    | 2.223 (6.0×)    |
| Tier 4 (C2)                              | <b>0.089</b>   | 0.083         | 0.501 (5.6×)    | 0.493 (5.5×)    |
| <i>Taglia <math>N = 1\,000</math>:</i>   |                |               |                 |                 |
| Tier 0 (Interprete puro)                 | <b>52.12</b>   | 110.64 (2.1×) | 125.72 (2.4×)   | 374.04 (7.2×)   |
| Tier 1 (C1)                              | <b>3.68</b>    | 3.88          | 8.71 (2.4×)     | 18.29 (5.0×)    |
| Tier 4 (C2)                              | <b>1.49</b>    | 1.44          | 4.63 (3.1×)     | 4.82 (3.2×)     |
| <i>Taglia <math>N = 100\,000</math>:</i> |                |               |                 |                 |
| Tier 0 (Interprete puro)                 | <b>5571.19</b> | 11540.42      | 12488.01        | 37431.55        |
| Tier 1 (C1)                              | <b>1172.27</b> | 1239.64       | 1575.74 (1.34×) | 1851.88 (1.58×) |
| Tier 4 (C2)                              | <b>799.50</b>  | 758.64        | 1261.43 (1.58×) | 655.42 (0.82×)  |

I risultati sperimentali mettono in luce considerazioni teoriche e pratiche fondamentali:

1. **Efficienza a Tier 0 (Interprete puro):** in modalità interpretata, `FusedStream` risulta oltre 2 volte più veloce perfino del ciclo Java manuale ( $52.12\mu s$  contro  $110.64\mu s$  su  $N = 1\,000$ , e  $5.57\text{ ms}$  contro  $11.54\text{ ms}$  su  $N = 100\,000$ ). Nel loop Java manuale ogni iterazione esegue una chiamata virtuale interpretata (`list.get(i)` con `invokevirtual`), mentre `FusedStream` accede direttamente all'array sottostante (`ArrayListAccessor`) tramite istruzioni `aaload` dirette. Rispetto a Java Stream ( $125.72\mu s$ ) e Scala View ( $374.04\mu s$ ), `FusedStream` è fino a 7.2 volte più rapido, dimostrando l'efficacia dell'inlining a compile-time anche in totale assenza di compilazione JIT.
2. **Efficienza immediata a Tier 1:** anche quando il JIT C2 è disabilitato e il compilatore C1 non esegue profilazione né ottimizzazioni complesse (Tier 1), `FusedStream` genera codice con prestazioni sovrapponibili al ciclo Java manuale ( $3.68\mu s$  contro  $3.88\mu s$  per  $N = 1\,000$ ). Al contrario, Java Stream e Scala View registrano tempi di esecuzione da  $2.4\times$  a  $5\times$  superiori, poiché dipendono strettamente da C2 per tentare di eliminare l'overhead degli oggetti intermedi e delle lambda.
3. **Azzeramento del warm-up:** poiché la fusione, la beta-reduction e l'eliminazione delle closure avvengono a tempo di compilazione tramite macro Scala 3, il bytecode emesso possiede già la forma ideale di un ciclo imperativo sequenziale. `FusedStream` non richiede quindi una fase di riscaldamento (*warm-up*) della JVM per eliminare le astrazioni funzionali, garantendo fin dal primo avvio prestazioni consistenti e deterministiche.

## 7.2.5 Impatto delle closure: il caso delle Capturing Lambdas

Nei microbenchmark sintetici le funzioni lambda impiegate sono tipicamente *non-capturing*, ovvero non fanno riferimento a variabili esterne allo scope locale della funzione stessa (come nel caso di una costante `t => t.amount > 50.0`). In questa situazione la JVM sfrutta l'infrastruttura di `LambdaMetafactory` per generare un unico singleton statico all'avvio, riutilizzandolo in ogni successiva invocazione ed eludendo qualsiasi costo di allocazione.

Nelle applicazioni reali i carichi di lavoro sono tuttavia parametrizzati: soglie di prezzo, scaglioni di sconto, aliquote IVA e identificatori utente variano dinamicamente e vengono letti dallo stato dell'applicazione o ricevuti come argomenti. In questi casi la funzione lambda diventa a tutti gli effetti una **capturing lambda** (o *closure*), poiché deve incorporare lo stato dell'ambiente lessicale circostante.

L'impatto sul runtime JVM è drastico:

- In Java Stream e Scala View la JVM non può riciclare un'istanza condivisa: a ogni singola esecuzione della query deve allocare sullo heap un nuovo oggetto closure contenente i campi per le variabili catturate;
- A Tier 0 (Interprete puro) e Tier 1 (C1), dove l'analisi di escape non è attiva, ogni invocazione genera allocazioni incontrollate nello spazio Eden e chiamate virtuali tramite `invokevirtual`;
- Persino a Tier 4 (C2 completo), all'interno di un'applicazione reale con molteplici query eterogenee (call site megamorfici), l'ottimizzatore JIT non riesce a eliminare l'allocazione delle closure tramite *Escape Analysis*, penalizzando il throughput e innescando frequenti cicli di garbage collection.

Al contrario, l'approccio di `FusedStream` azzerà alla radice il problema. La macro espande il corpo della lambda mediante *beta-reduction* direttamente all'interno dell'AST del ciclo `while`: le variabili esterne catturate non richiedono alcun oggetto wrapper di supporto, ma vengono lette come semplici variabili locali già allocate sullo stack frame.



Per quantificare sperimentalmente questo fenomeno, le 8 query eterogenee dello scenario megamorfo sono state rieseguite trasformando tutte le costanti in variabili catturate dall'ambiente locale circostante. La Tabella 7.6 riporta i tempi medi di esecuzione a confronto sui tre livelli di compilazione JIT (Tier 0, Tier 1 e Tier 4).

Tabella 7.6: Confronto prestazioni con Capturing Lambdas sui livelli JIT ( $\mu\text{s/op}$ ).

| Configurazione JIT        | FusedStream    | Java Manual             | Java Stream               | Scala View                |
|---------------------------|----------------|-------------------------|---------------------------|---------------------------|
| <i>Taglia N = 10:</i>     |                |                         |                           |                           |
| Tier 0 (Interprete puro)  | <b>1.006</b>   | 1.164 (1.16 $\times$ )  | 6.907 (6.87 $\times$ )    | 15.922 (15.83 $\times$ )  |
| Tier 1 (C1)               | <b>0.043</b>   | 0.038                   | 0.464 (10.79 $\times$ )   | 0.667 (15.51 $\times$ )   |
| Tier 4 (C2 completo)      | <b>0.011</b>   | 0.010                   | 0.110 (10.00 $\times$ )   | 0.089 (8.09 $\times$ )    |
| <i>Taglia N = 100:</i>    |                |                         |                           |                           |
| Tier 0 (Interprete puro)  | <b>5.700</b>   | 10.399 (1.82 $\times$ ) | 18.345 (3.22 $\times$ )   | 50.282 (8.82 $\times$ )   |
| Tier 1 (C1)               | <b>0.381</b>   | 0.373                   | 1.228 (3.22 $\times$ )    | 2.307 (6.05 $\times$ )    |
| Tier 4 (C2)               | <b>0.098</b>   | 0.085                   | 0.574 (5.86 $\times$ )    | 0.640 (6.53 $\times$ )    |
| <i>Taglia N = 1000:</i>   |                |                         |                           |                           |
| Tier 0 (Interprete puro)  | <b>52.62</b>   | 104.47 (1.99 $\times$ ) | 128.04 (2.43 $\times$ )   | 379.70 (7.22 $\times$ )   |
| Tier 1 (C1)               | <b>3.83</b>    | 3.77                    | 9.25 (2.41 $\times$ )     | 18.49 (4.83 $\times$ )    |
| Tier 4 (C2)               | <b>1.66</b>    | 1.54                    | 5.36 (3.23 $\times$ )     | 5.92 (3.56 $\times$ )     |
| <i>Taglia N = 100000:</i> |                |                         |                           |                           |
| Tier 0 (Interprete puro)  | <b>5905.34</b> | 10822.35                | 12829.36 (2.17 $\times$ ) | 38404.56 (6.50 $\times$ ) |
| Tier 1 (C1)               | <b>1236.95</b> | 1268.68                 | 1681.90 (1.36 $\times$ )  | 1826.99 (1.48 $\times$ )  |
| Tier 4 (C2)               | <b>923.07</b>  | 978.49                  | 1382.83 (1.50 $\times$ )  | 721.10 (0.78 $\times$ )   |

I dati evidenziano tre aspetti fondamentali:

1. **Decadimento prestazionale di Java Stream e Scala View:** rispetto alla versione non-capturing (Tabella 7.5), Java Stream subisce un incremento sensibile dei tempi su tutti i livelli JIT. Su  $N = 10$ , dove il calcolo sui dati è minimo e l'allocazione delle closure ha il massimo peso relativo, il tempo medio di Java Stream aumenta del +42.7% a Tier 0 (da  $4.84 \mu\text{s}$  a  $6.91 \mu\text{s}$ ), del +40.2% a Tier 1 (da  $0.331 \mu\text{s}$  a  $0.464 \mu\text{s}$ ) e del +26.4% a Tier 4 (da  $0.087 \mu\text{s}$  a  $0.110 \mu\text{s}$ ). Persino su grandi volumi ( $N = 100000$ ), Java Stream rallenta di quasi il 10% a Tier 4 (da  $1261.43 \mu\text{s}$  a  $1382.83 \mu\text{s}$ ), attestandosi a un distacco di  $10\times$  rispetto a FusedStream sui piccoli campioni e di  $1.5\times$  sulle grandi taglie.
2. **Totale immunità di FusedStream:** i tempi misurati per FusedStream con capturing lambdas sono identici a quelli ottenuti con lambdas non-capturing (Tabella 7.5). Poiché la compilazione tramite macro fonde direttamente i predicati nel corpo del ciclo, l'accesso a variabili esterne viene compilato come accesso diretto a variabili locali sullo stack. Non viene allocato alcun oggetto closure, annullando sul nascere l'overhead a runtime.
3. **Solidità a Tier 0:** in modalità interpretata pura (Tier 0), FusedStream mantiene il suo netto vantaggio, risultando da  $6.8\times$  a  $15.8\times$  più veloce delle librerie standard sulle piccole taglie, e quasi 2 volte più veloce del loop Java manuale sui grandi volumi ( $5.91 \text{ ms}$  contro  $10.82 \text{ ms}$ ) per via dell'accesso vettoriale diretto alla memoria (aaload).

### 7.2.6 Allocazioni e comportamento del Garbage Collector

Oltre al throughput, un fattore critico per le prestazioni stabili in produzione è l'impatto sul Garbage Collector (GC). Le librerie funzionali convenzionali producono un flusso continuo di oggetti a vita breve:

- **Istanze di closure:** come mostrato nella Sezione 7.2.5, ogni lambda che cattura variabili dall'ambiente circostante alloca una nuova istanza sullo heap a ogni iterazione o invocazione della pipeline.
- **Descrittori di pipeline:** ogni stadio intermedio (`map`, `filter`) in Java Stream alloca un nodo `ReferencePipeline` e le relative classi interne di `Sink`.
- **Boxing di primitivi:** in assenza di tipi specializzati (`DoubleStream`), ogni valore transita attraverso wrapper ad oggetti (`Double`).
- **Wrapper di iteratori:** in Scala View ogni operatore crea un nuovo oggetto `Iterator` anonimo che incapsula l'iteratore a monte.

Questo ricambio continuo carica la regione *Eden* della Young Generation, inducendo frequenti cicli di *Minor GC* e fluttuazioni nella latenza di risposta.

`FusedStream` adotta una strategia a **zero allocazioni nello stato stazionario**:

- Il ciclo di attraversamento è un costrutto nativo `while` senza iteratori intermedi;
- I valori trasformati transitano come registri locali sullo stack JVM;
- I collector primitivi (come `summing`) accumulano su variabili primitive (`double`, `int`), eliminando completamente il boxing;
- L'accesso a `ArrayList` estrae gli elementi direttamente dall'array sottostante tramite `ArrayListAccessor`.

Durante l'esecuzione di una pipeline sequenziale, `FusedStream` non alloca memoria nello heap, assicurando tempi deterministici e totale assenza di pause introdotte dal Garbage Collector.

# Capitolo 8

## Retrospettiva

In questo capitolo vengono riassunte le considerazioni emerse durante lo sviluppo di `FusedStream`, analizzando i compromessi tecnici adottati, i limiti attuali della soluzione e le prospettive di evoluzione futura.

### 8.1 Limiti e compromessi progettuali

La progettazione di un motore di fusione a compile-time su JVM impone un bilanciamento continuo tra espressività dell'API, complessità delle macro e prestazioni del codice generato. Le principali scelte di compromesso hanno riguardato:

- **Modello push ed esclusione dello zip:** l'adozione dell'approccio push ha permesso di eliminare ogni iteratore a runtime e di fondere le operazioni in loop imperativi compatti. Di contro, non è possibile supportare direttamente lo *zipping* tra stream concorrenti senza reintrodurre strutture di buffering o coroutine, che contrasterebbero con l'obiettivo di azzerare le allocazioni.
- **Parallelismo su collezioni indicizzate:** la scelta di riservare `parallel()` alle collezioni indicizzate a dimensione nota (`Array`, `ArrayList`) impedisce rallentamenti su sorgenti inadatte. D'altro canto, esclude la parallelizzazione di flussi basati su iteratori.
- **Complessità della metaprogrammazione:** l'uso avanzato di macro e beta-reduction sposta l'intero onere sul tempo di compilazione. Il codice generato azzerava l'overhead a runtime, ma i tempi di compilazione e la complessità interna risultano superiori rispetto a una libreria a runtime.

### 8.2 Sviluppi futuri

Le evidenze raccolte durante la validazione sperimentale aprono interessanti direzioni di miglioramento:

- **Ottimizzazione dello scheduler parallelo:** come illustrato nella Sezione [7.2.3](#), Java Parallel Stream sfrutta `CountedCompleter` e ottiene maggiore efficienza sulle taglie intermedie. L'attuale architettura di `FusedStream` si appoggia a un `ExecutionContext` standard sincronizzato via `CountDownLatch`. Un'indagine su uno scheduler dedicato basato direttamente su primitive `ForkJoinPool` costituisce una priorità per abbattere l'overhead di sottomissione; tale approfondimento non è stato possibile nell'ambito di questa tesi per motivi di tempo.

- **Vettorizzazione SIMD:** con le nuove API del JDK (Project Panama e la *Vector API*), il generatore potrebbe emettere istruzioni vettoriali esplicite per le trasformazioni e le riduzioni aritmetiche su tipi primitivi, superando i limiti dell'auto-vettorizzazione di HotSpot C2.
- **Estensione dei collector paralleli:** introduzione di collector paralleli complessi con partizionamento hash (quali `groupBy` o aggregazioni associative multi-chiave), progettati per limitare le contese senza creare collezioni intermedie temporanee.

## 8.3 Conclusioni

Il lavoro svolto ha dimostrato la fattibilità e l'efficacia di un'architettura di *stream fusion* a tempo di compilazione in Scala 3. Sfruttando le macro tipizzate, la reflection a compile-time e l'inlining delle lambda, `FusedStream` riesce a colmare il divario storico tra la chiarezza dichiarativa della programmazione funzionale e la velocità dei loop imperativi scritti a mano su JVM.

La valutazione empirica ha confermato che, mentre le soluzioni esistenti degradano sensibilmente in scenari applicativi reali per via del fallimento dell'inline caching sui call site megamorfici, la fusione a compile-time mantiene prestazioni invarianti e competitive con il codice Java manuale a zero allocazioni.

# Bibliografia

- [1] Duncan Coutts, Roman Leshchinskiy, and Don Stewart. Stream fusion: From lists to streams to nothing at all. *ACM SIGPLAN Notices*, 42(9):315–326, 2007.
- [2] Oleg Kiselyov, Aggelos Biboudis, Nick Palladinis, and Yannis Smaragdakis. Stream fusion, to completeness. In *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages*, pages 285–299, 2017.
- [3] Dmitry Konstantinov. CASSANDRA-21289: Avoid capturing lambda allocation in UnfilteredSerializer.deserializeRowBody. Apache Cassandra Issue Tracker, 2026. Accessed: 2026-09-27.
- [4] Dmitry Konstantinov. CASSANDRA-21588: Stop using java streams in CIDRPermissions.SubsetPermissions#canAccessFrom hot method. Apache Cassandra Issue Tracker, 2026. Accessed: 2026-09-27.
- [5] Shayan Najd and Simon Peyton Jones. Trees that grow. *Journal of Universal Computer Science*, 23(1), 2017.
- [6] OpenJDK. Java Microbenchmark Harness (JMH). <https://github.com/openjdk/jmh>, 2024.
- [7] John Rose and Project Valhalla Team. The saga of the parametric vm. Technical report, OpenJDK Project Valhalla, 2021. June 2020–April 2021.
- [8] Nicolas Stucki, Jonathan Immanuel Brachthäuser, and Martin Odersky. Multi-stage programming with generative and analytical macros. In *Proceedings of the 20th ACM SIGPLAN International Conference on Generative Programming: Concepts and Experiences*, pages 110–122, 2021.
- [9] Nicolas Alexander Stucki. *Scalable metaprogramming in scala 3*. PhD thesis, EPFL, 2023.